

AI Agents

50

Real-World

Best Practices

That Actually Work

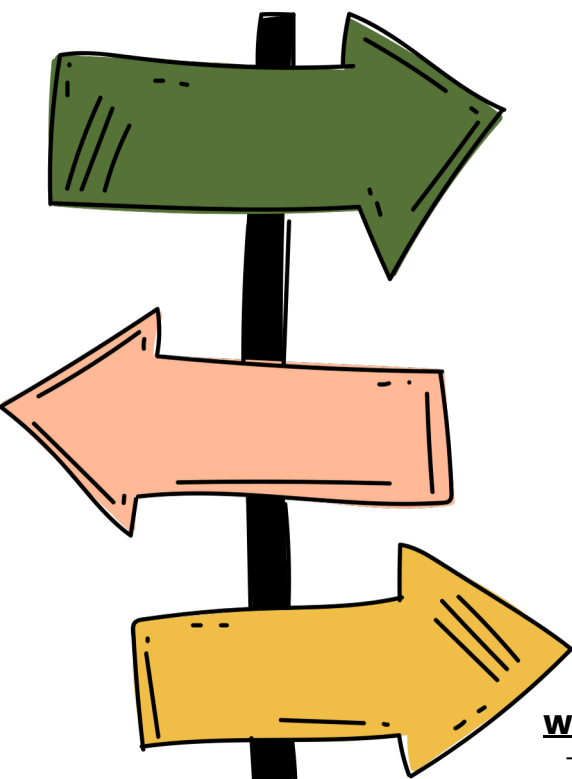


by: Dr. Maryam Miradi
Built from 20+ years in AI.
Field-tested. Zero fluff.

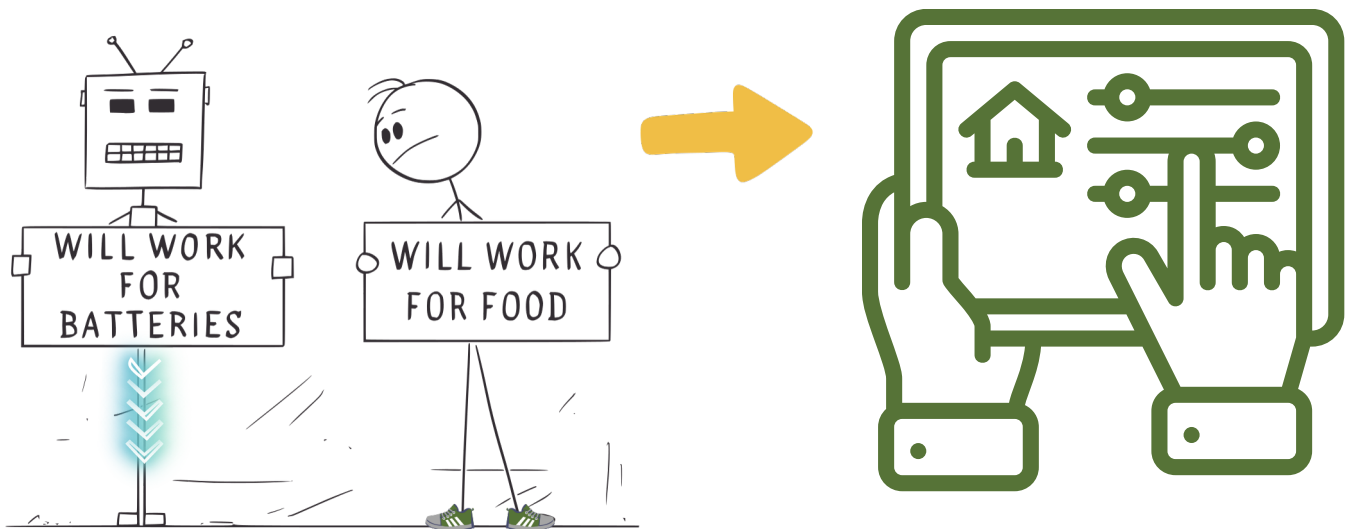


“Over the past year, I’ve built over 400+ AI agents—for my training programs and real-world deployments across multiple industries. With 20+ years of hands-on AI experience, I’ve seen exactly what separates agents that just run from those that deliver real results. This guide distills 50+ best practices, a blend of deep technical insight and field-tested shortcuts to help you build real-world AI agents that actually work. No fluff. Just Progress.”

–Dr. Maryam Miradi

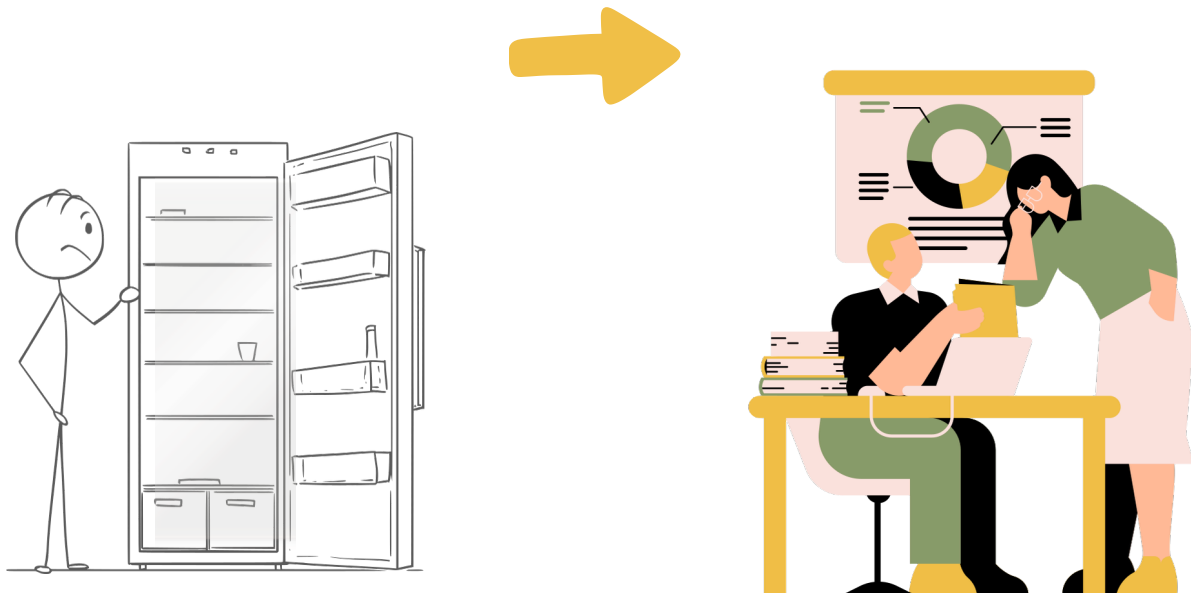


1 AI Agents Are Not Your Employees



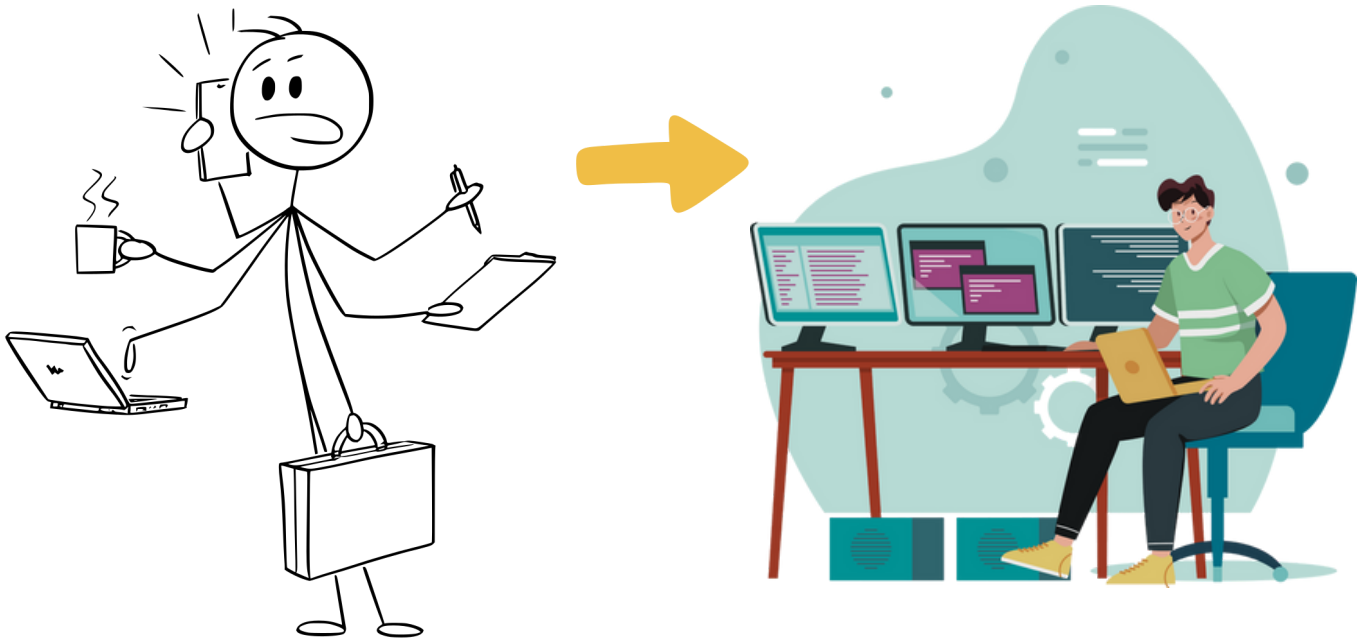
It's a common misconception to label AI agents as automations or even "your own employees." In reality, AI agents sit somewhere in between. Unlike traditional automations—where every step is hardcoded and predetermined—agents require detailed training to perform tasks. They lack the full autonomy of human employees; in fact, one employee can typically handle multiple standard operating procedures (SOPs), while one AI agent is best suited to managing one SOP at a time. This difference means you may need many more agents to cover the same breadth of tasks a human team could handle.

2 Start from Well-Documented Processes



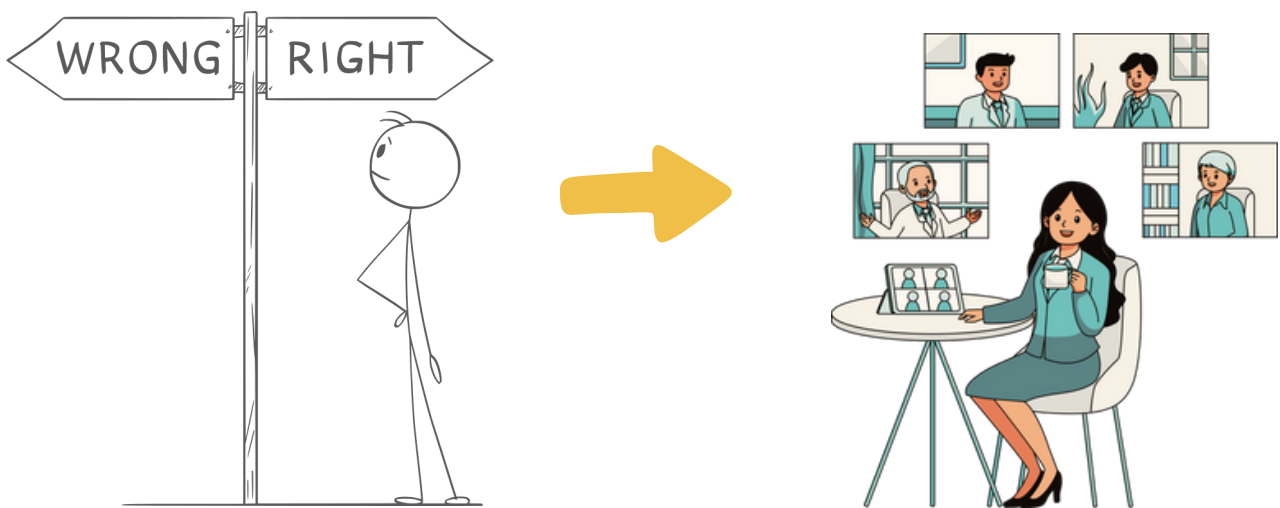
Before you train an AI agent, look for existing, well-documented processes—your SOPs. These documents outline the exact steps employees follow in a business and serve as the perfect foundation for teaching an agent how to operate. Using clear SOPs simplifies the training process and eliminates the need for endless client queries. In essence, well-documented processes are your blueprints for building reliable AI agents.

3 Business Owners Will Never Build Their Own Agents



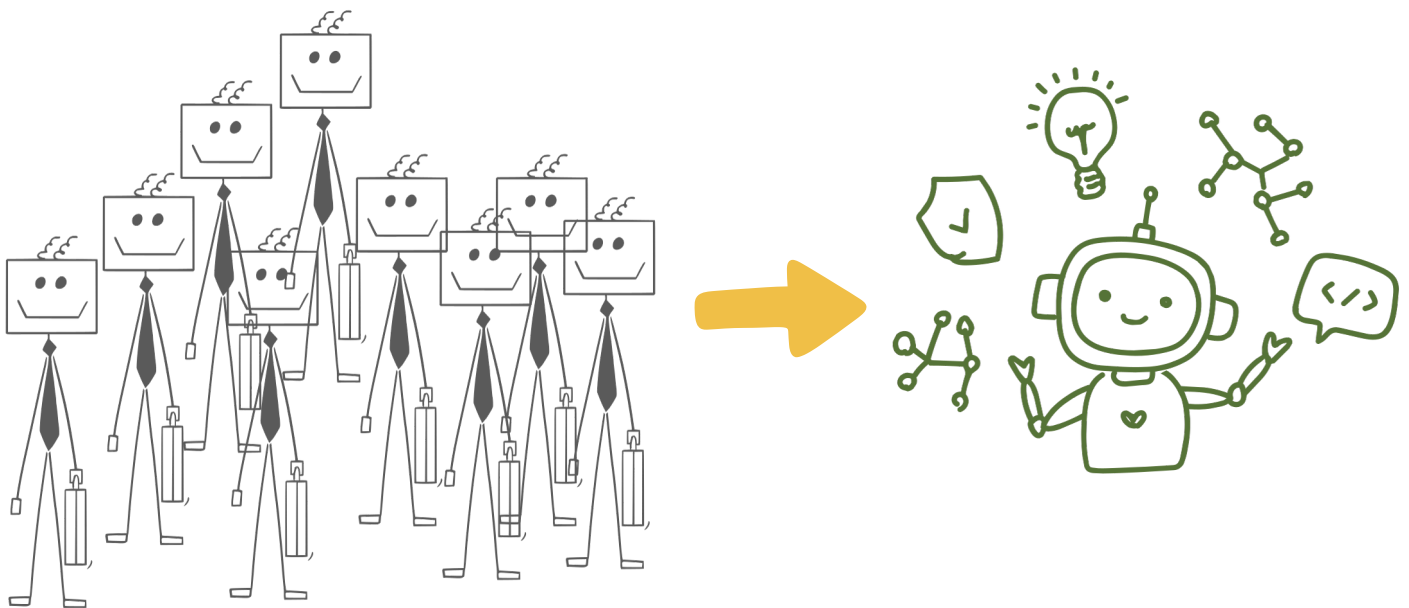
Even as technologies evolve and agents begin to generate other agents from simple prompts, business owners will still rely on expert developers. Just as no-code tools didn't eliminate the need for skilled developers, AI agent platforms won't end the demand for AI agent expertise. Determining the right agents to build—tailored to a business's unique needs—remains a specialized skill that clients will always entrust to professionals.

4 Business Owners Have No Idea Which Agents They Need



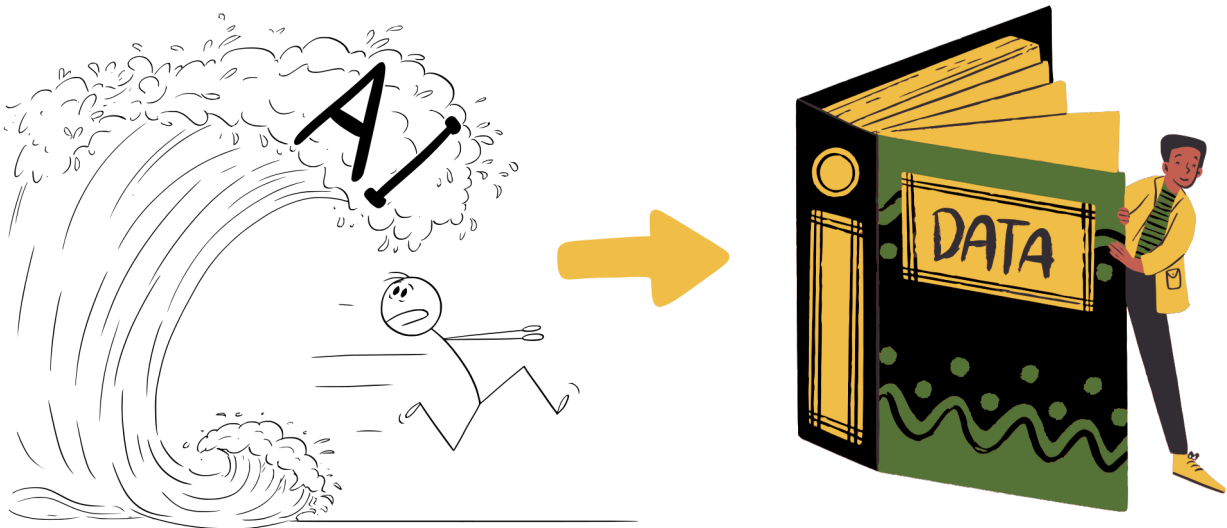
When clients first approach you, they often have a rough idea of what they want. However, in about half of the cases, their initial suggestions are far from the most valuable solutions. Effective consulting involves mapping out the customer journey—often using tools like Figma—to pinpoint exactly where automation can bring the most value. Use client ideas as feedback rather than a definitive blueprint, and focus on uncovering hidden opportunities in their processes.

5 You Don't Need 20 Plus Agents



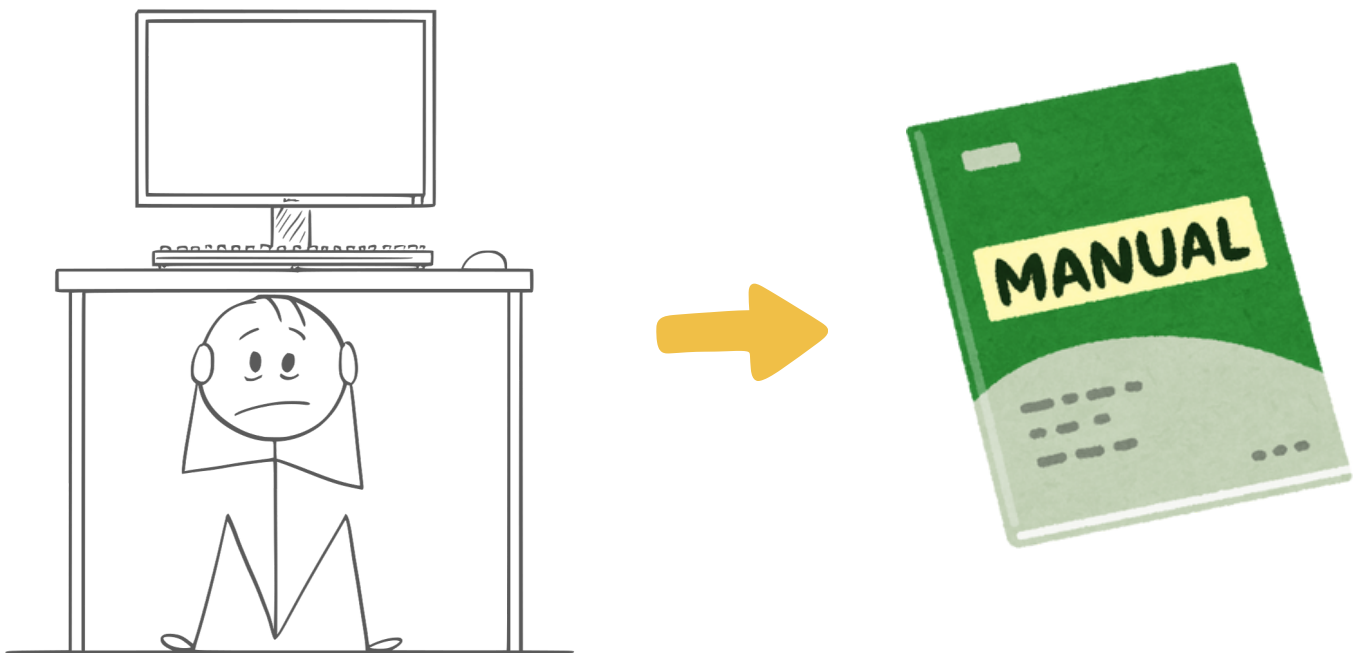
A common trend is to build as many agents as possible, but more isn't always better. Flooding your system with numerous agents can make maintenance and debugging a nightmare. It also increases operational costs and slows down response times. Start with the smallest viable agent that delivers measurable value. Once that agent is fine-tuned and deployed successfully, you can incrementally add more agents where needed. This is the approach that you will learn also in my **AI Agents Training**.

6 Data-Driven Decisions with Actions Deliver Results



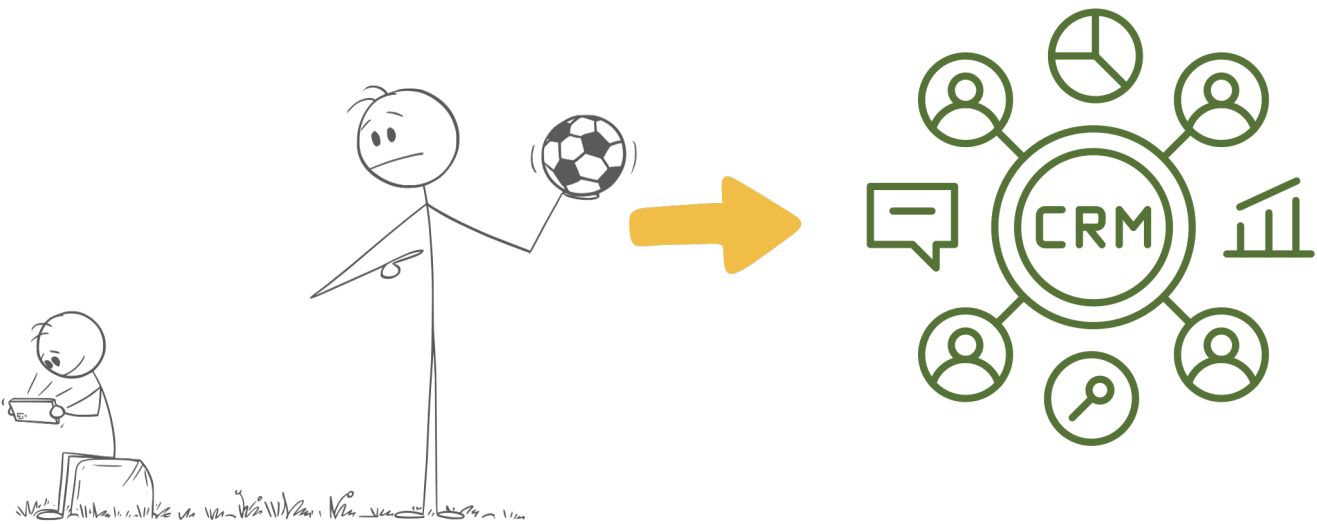
There's an old saying in data science—"garbage in, garbage out." The same holds true for AI agents. However, the true power comes not just from data, but from combining that data with the right actions. For example, an agent that understands both the data behind effective Facebook marketing campaigns and can control the Facebook API will outperform one that only processes data. Integrating actionable insights into your agent's design is key to driving tangible improvements.

7 Prompt Engineering Is an Art



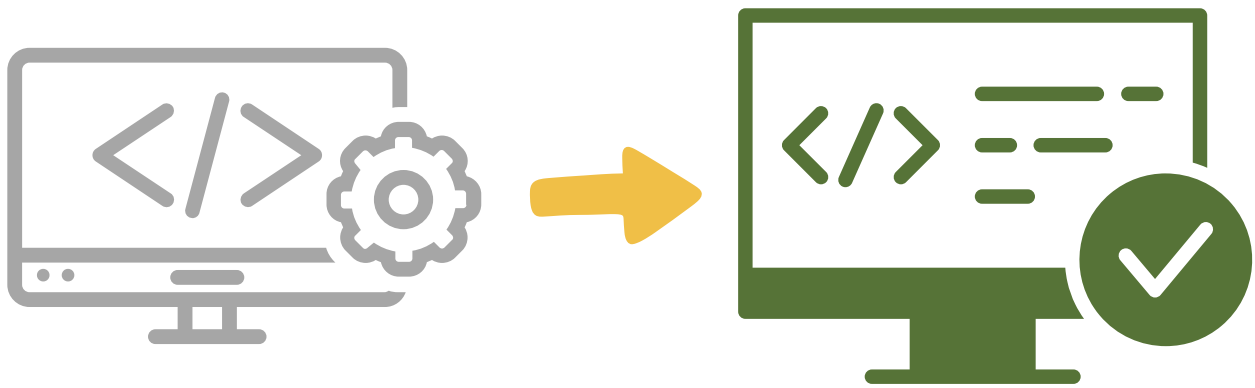
Prompt engineering has evolved into a critical discipline in the AI world. Crafting effective prompts requires the same care as writing a thoughtful essay or blog post. It's not enough to provide a simple instruction; you must offer multiple examples, pay attention to the order of information, and continuously iterate and test your prompts. Small changes in wording or structure can transform an agent from unreliable to consistently effective. If you are interested in advanced topics of Prompt Engineering, like **Prompt Tuning and Prefix Tuning, Watch this 9-min Free Tutorial.**

8 Integrations Are Just as Important as Functionality



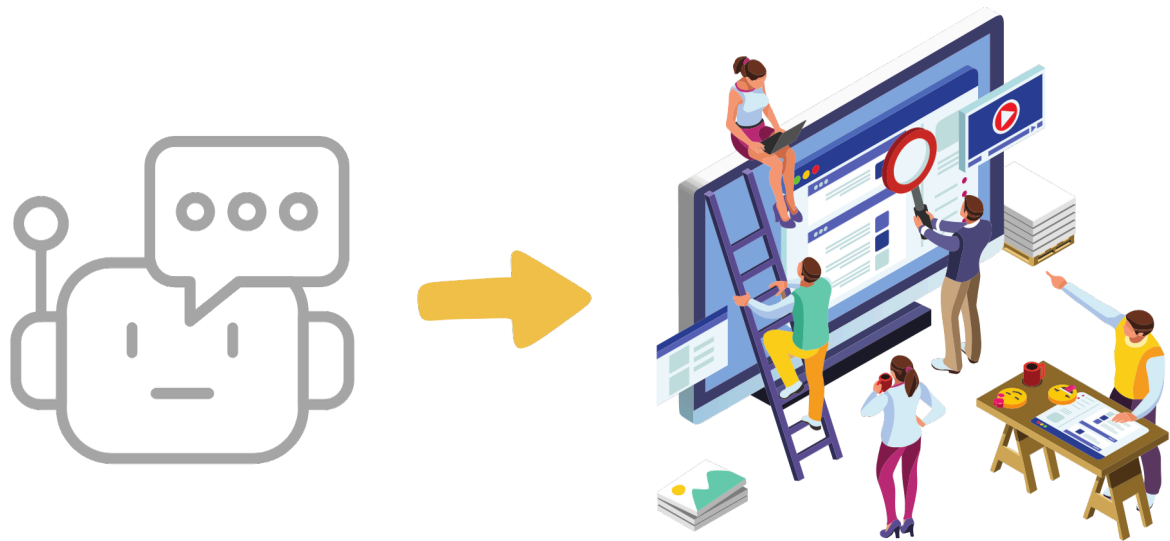
While it's tempting to focus solely on an agent's internal capabilities, integrations with existing systems are equally crucial. If your AI agent isn't seamlessly embedded into the systems your employees use every day, even the most powerful agent won't deliver its full value. For example, a customer support agent should integrate with the same CRM or ticketing system (like Zendesk) that the support team uses to ensure a smooth workflow.

9 Agent Reliability Has Been Solved



Many startups worry about agent reliability, but in our experience, if an agent isn't performing reliably, the fault lies with the developer—not the agent itself. By using robust data validation libraries (such as pydantic) to validate inputs and outputs, you can effectively prevent major errors. The lesson here is clear: build your agents with rigorous validation to ensure that they act within safe, predetermined parameters. Watch this Youtube Video on **Pydantic is all you Need**.

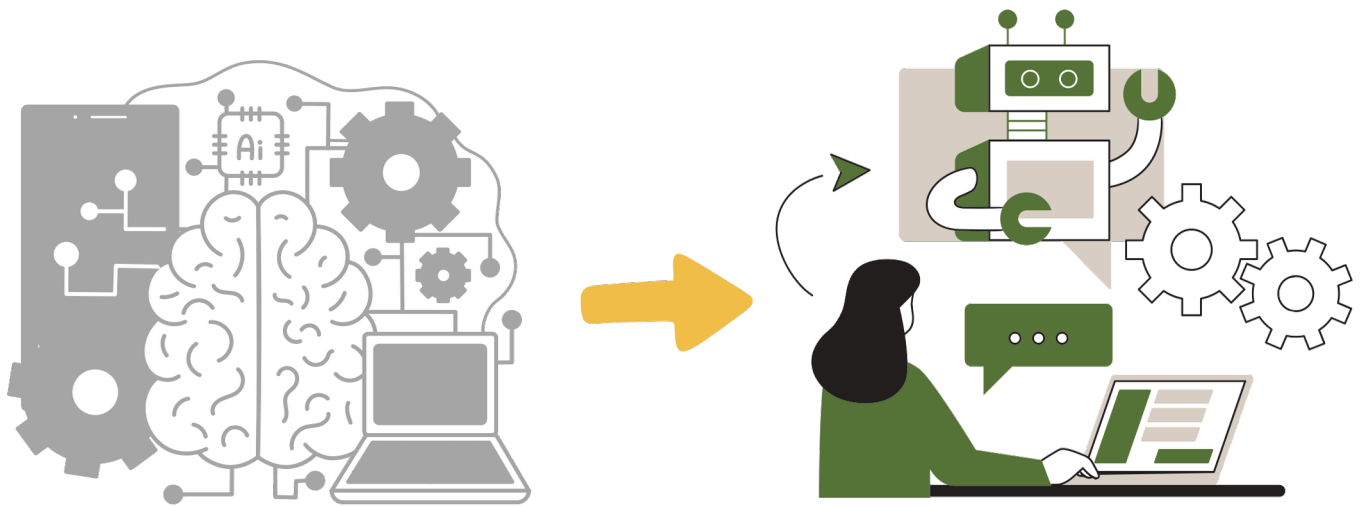
10 Tools Are the Most Important Component



When building AI agents, remember that instructions, knowledge, and actions form the three pillars of success. Surprisingly, about 70% of our work focuses on building actions—the tools through which agents deliver value. Unlike traditional chatbots that generate value through responses, AI agents execute tasks. Build and structure your tools well, and you can create agents capable of handling a wide range of use cases. In my AI Agents Training, I teach you **how to build plenty of Tools to build and use for AI Agents**.

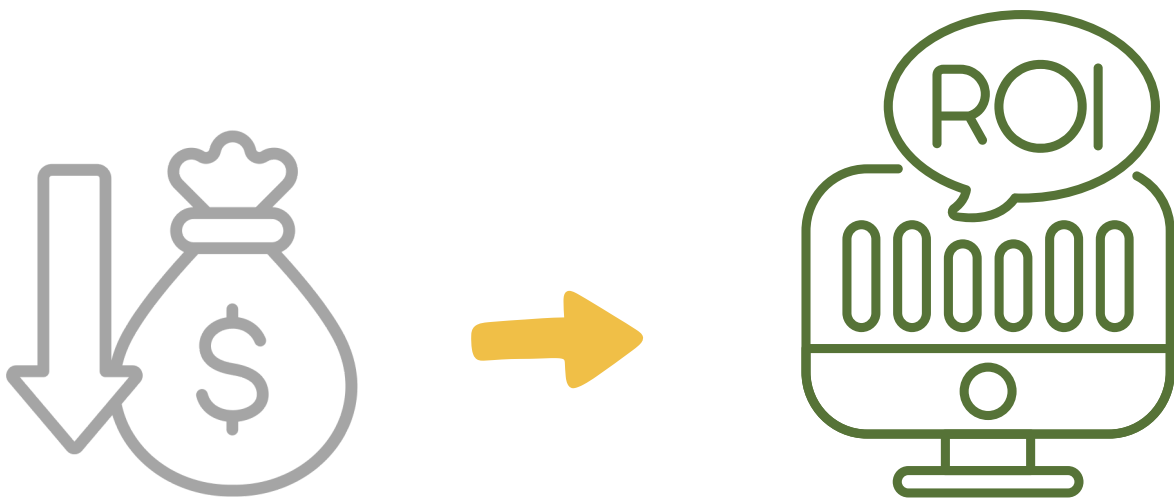
11

No More Than Four to Six Tools Per Agent



A practical rule of thumb in our agency is to limit each agent to four to six tools. Overloading an agent with too many tools can lead to hallucinations or confusion about which tool to use and in what order. If you notice performance issues, consider splitting the functionality across multiple agents to maintain clarity and reliability. During **my training** you will see how keeping the number of Tools can give you a clear workflow.

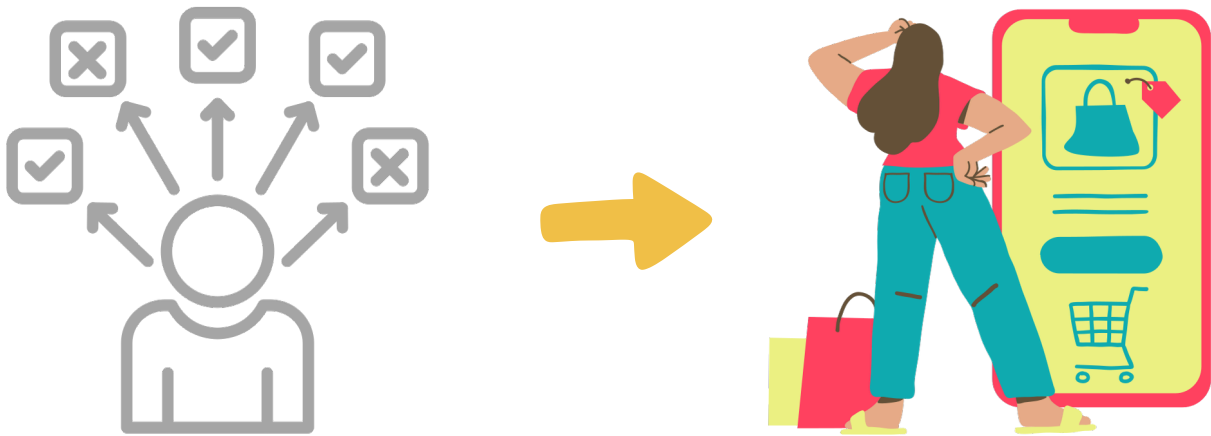
12 Model Costs Don't Matter



While model costs often make headlines, our experience has taught us that they are a minor consideration when the ROI is significant. If an AI agent delivers a tremendous return on investment compared to manual processes, the cost of the model becomes negligible. Focus on value creation, and the costs will naturally be justified by the substantial savings and efficiencies gained. I build my entire **OpenAI Swarm and CrewAI Training** with less \$20 API costs of OpenAI.

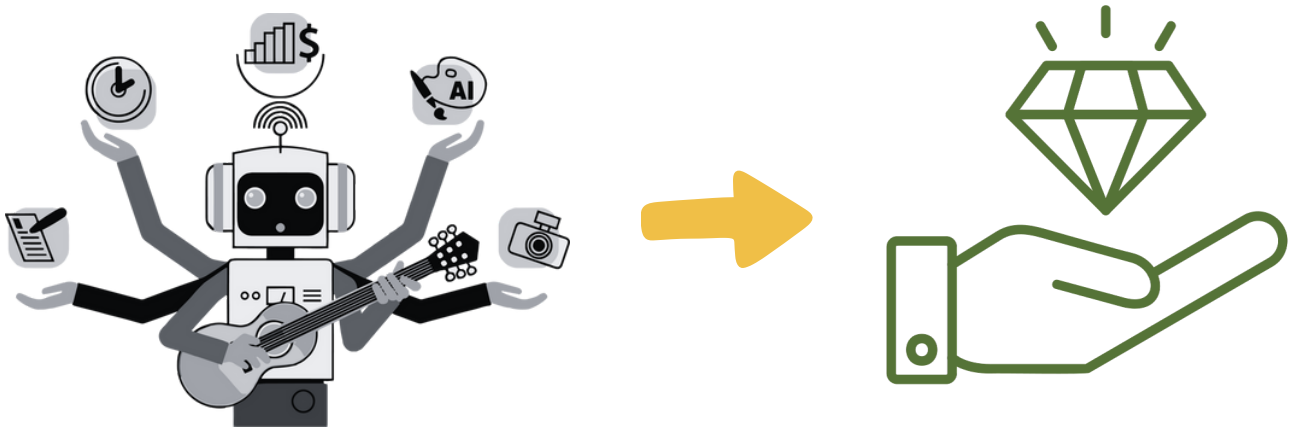
13

Clients Don't Care About Which Model You Use



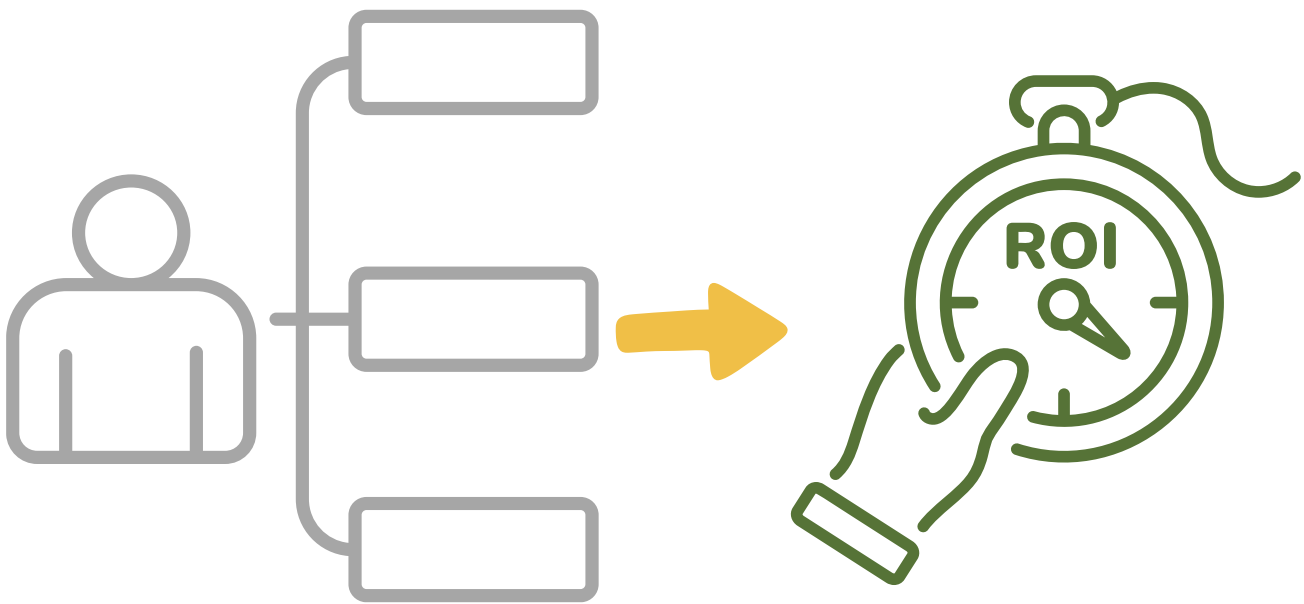
Surprisingly, most clients are indifferent about the underlying model you choose—as long as the solution meets their needs and adheres to data privacy requirements. Whether you use an open- source model or a private Azure OpenAI instance, what matters most is the value and reliability the agent delivers. This flexibility allows you to choose the model that best suits your development process and client policies. I have really good experience with Llama, DeepSeek and OpenAI.

14 Don't Automate Until Value Has Been Established



One of the biggest pitfalls is attempting to automate a process that hasn't yet proven its value. Some clients may be tempted to automate an untested process, but this approach is fraught with risk and high development costs. Instead, establish that a process works effectively through manual execution or a pilot test before investing in full automation with AI agents. In this free Tutorial, I show you how to build AI Agents in **Google Colab** without struggling weeks with Data Engineering. Fact is first see if your agents has business value then step into the deployment battle.

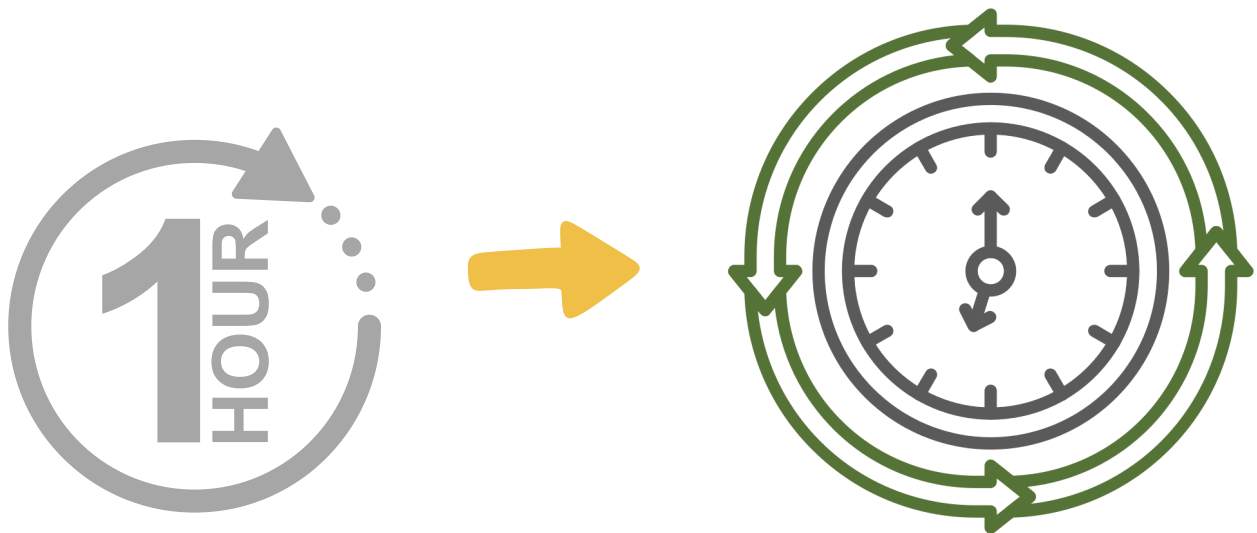
15 **Focus on ROI, Not Just Use Cases**



Rather than getting caught up in abstract use cases, always consider the return on investment. Our ROI formula factors in the cost of employee labor, operational expenses, and development costs. For example, if an employee costs \$50 per hour and spends 10 hours a week on a task, and your development costs are \$5,000, the ROI over a year could be as high as 5.6 times. This focus on measurable value ensures that your AI solutions drive real business benefits.

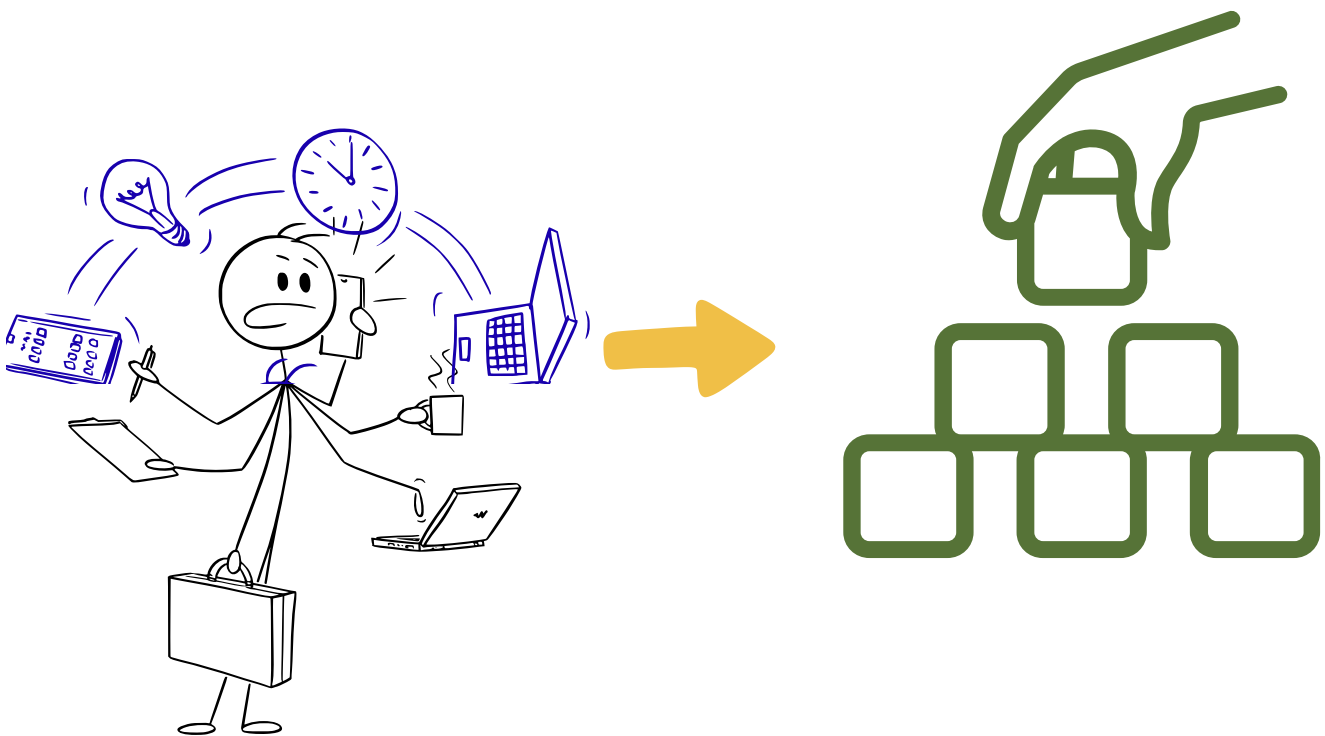
16

Agent Development Is an Iterative Process



Much like in data science competitions, building effective AI agents requires experimenting with multiple architectures and parameters. The more iterations you test, the clearer it becomes which design works best for a given solution. Don't be afraid to build several versions of an agent and refine them based on performance and real-world testing.

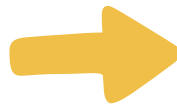
17 Use a Divide and Conquer Approach



Complex problems are best solved by breaking them down into manageable parts. In our development process, we often start by building a single, self-contained agent that can operate independently. Once this agent is validated, we can expand the system incrementally. This approach not only reduces risk but also ensures that each component delivers value before being integrated into a larger solution.

18

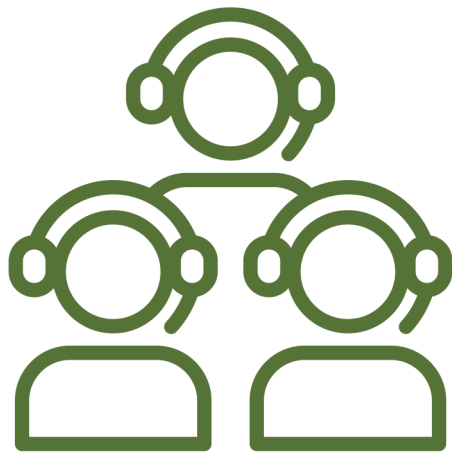
Evals Are Crucial for Large Enterprises



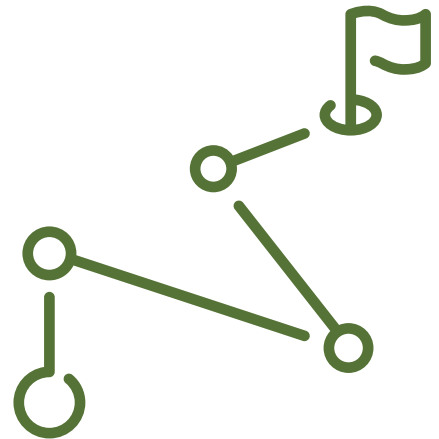
Evaluation metrics (evals) help track an agent's performance over time and are essential for continuous improvement. While small- and medium-sized businesses might not see significant benefits from evals due to lower request volumes, large enterprises can use them to refine solutions and maintain competitive advantage. For high-traffic environments, setting up robust evals from the start can be a game-changer.

19

Two Types of Agents—Agents and Workflows

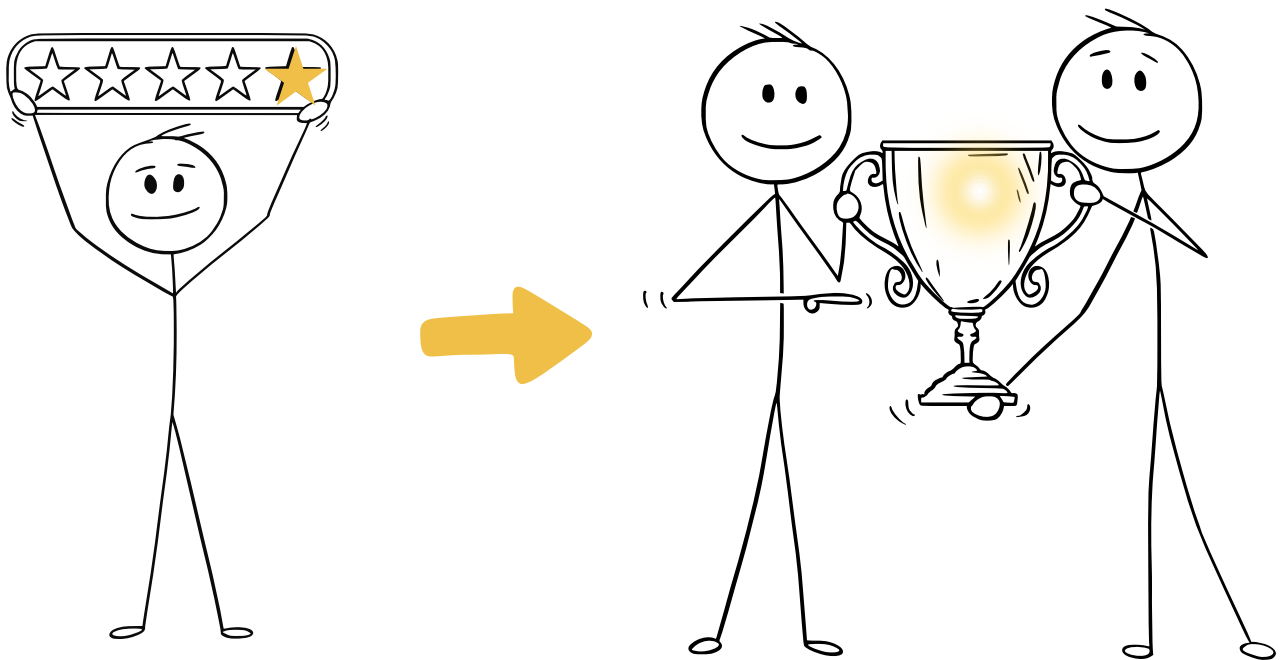


VS



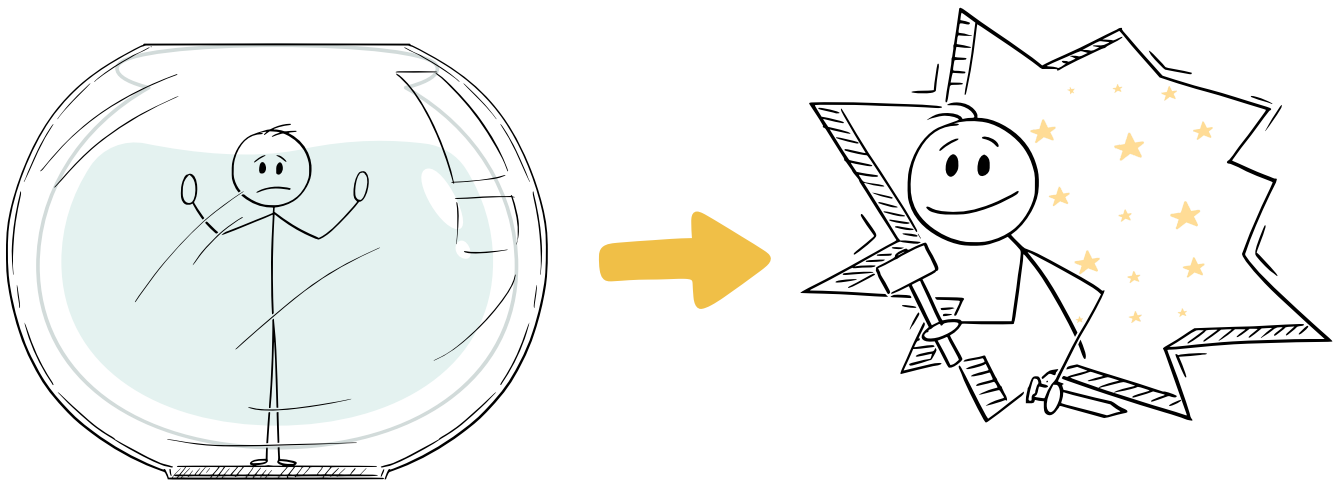
My experience revealed that not all processes should be automated with a fully agentic approach. Sometimes, a standard workflow—where the steps are predetermined and must be followed in a fixed sequence—is more appropriate. In these cases, the workflow can incorporate “agentic” steps to handle tasks that require a degree of autonomy. Recognizing the difference between pure agents and agentic workflows can help you choose the right strategy for each process.

20 Agents Need to Be Adaptable Based on Feedback



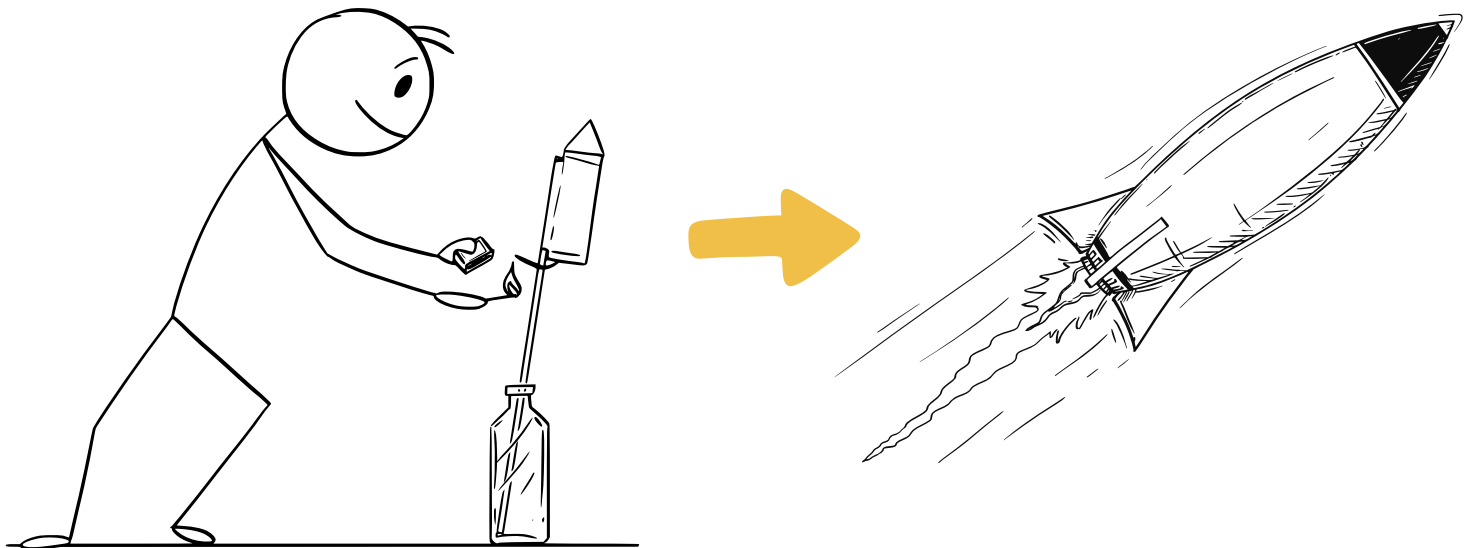
For an agent to be truly effective, it must not only interact with its environment but also adapt based on the feedback it receives. Ensure that your agents are equipped with tools not only to modify their environment but also to analyze the results of their actions. For instance, an agent updating a database should also verify that the update was successful, thereby closing the loop on its task.

21 Don't Build Around Limitations



Technology evolves rapidly. Building your agents around today's limitations can render your solutions obsolete tomorrow. For example, workarounds you create to manage context token limits may become unnecessary when new models are released. Instead, design your agents with a forward-thinking mindset, anticipating improvements and avoiding constraints that may soon vanish.

22 Deploying Agents Is Harder Than Building Them



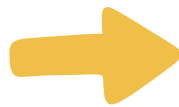
Technology evolves rapidly. Building your agents around today's limitations can render your solutions obsolete tomorrow. For example, workarounds you create to manage context token limits may become unnecessary when new models are released. Instead, design your agents with a forward-thinking mindset, anticipating improvements and avoiding constraints that may soon vanish.

23 Waterfall Projects Don't Work – Embrace Agility



Traditional waterfall project models are ill-suited for the dynamic nature of AI agent development. These projects evolve continuously, and new opportunities often emerge during the development cycle. Instead of locking yourself into rigid, long-term scopes, adopt a subscription-based, agile approach. By partnering with clients rather than acting as a one-time outsourced team, you can adapt to changing needs and deliver ongoing value.

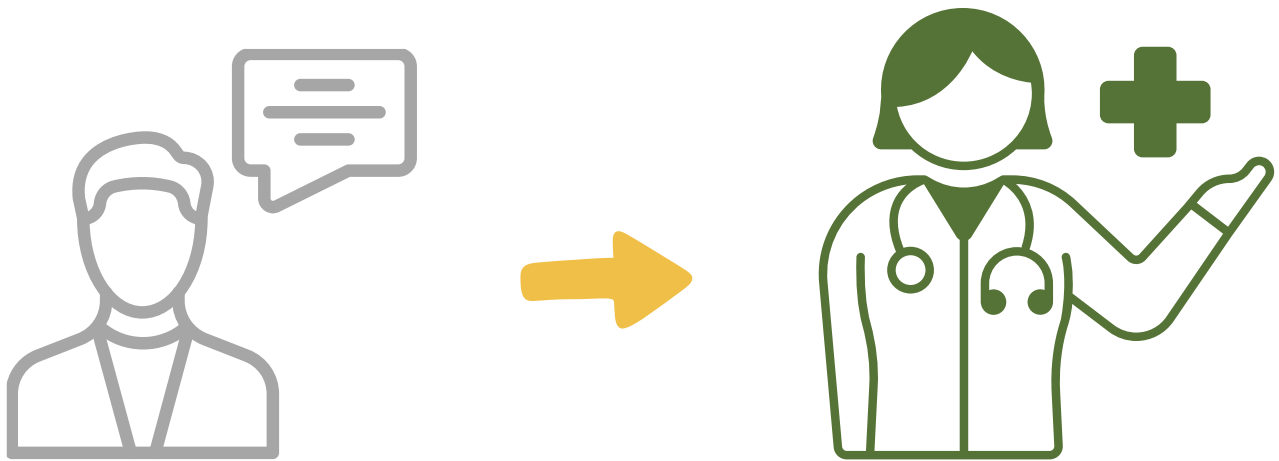
24 Include a Human in the Loop for Mission-Critical Agents



For mission-critical tasks where even a single error can have significant consequences, it's essential to include human oversight. Initially integrating a human review step—such as approving marketing campaigns before launch—can prevent costly mistakes. As the agent becomes more reliable and fine-tuned, you can gradually reduce human intervention, but never at the expense of safety and quality in critical operations.

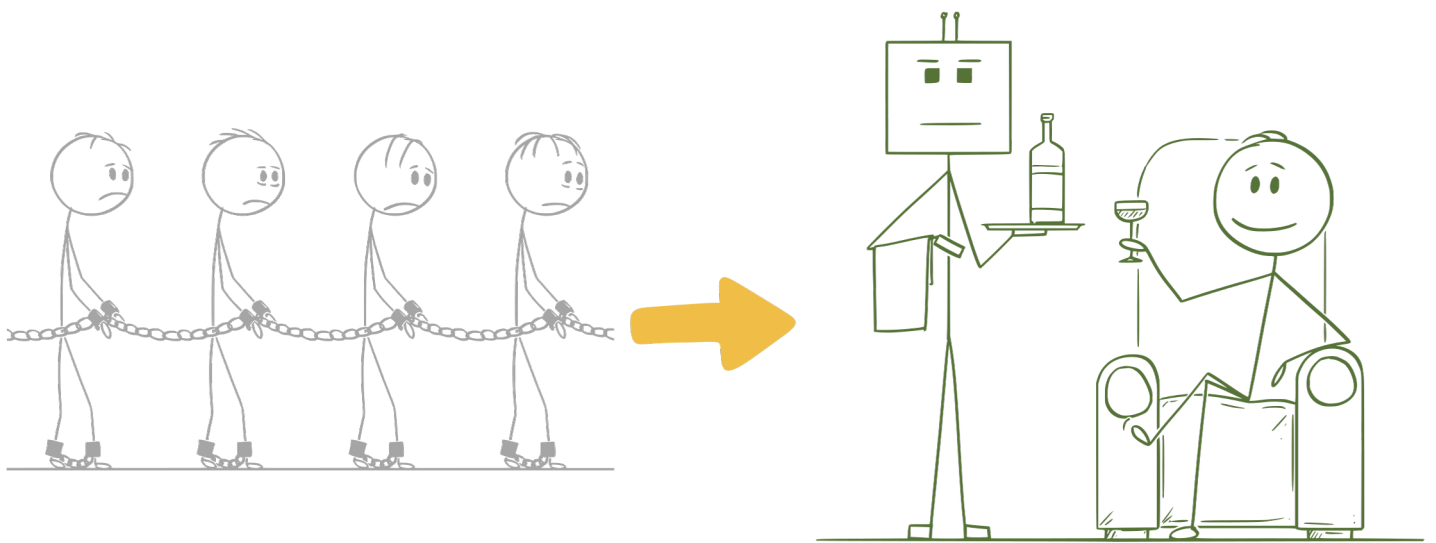
25

Vertical AI Agents Define the Future



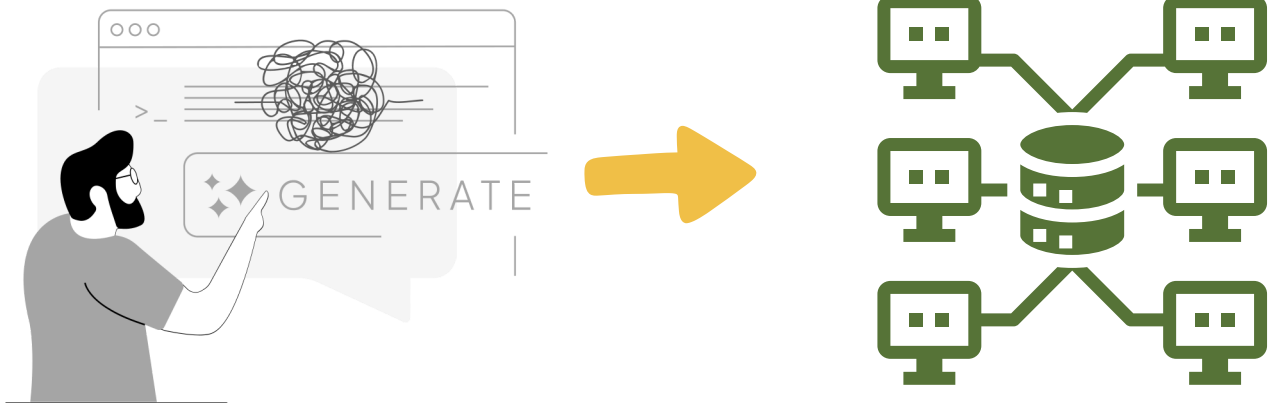
Vertical AI agents are specialized for a particular industry or use case, much like niche B2B SaaS solutions. While it's fine to start with horizontal agents that cater to a broad audience, the true scalability and profitability come from specializing. By understanding a customer's specific challenges in depth, you can fine-tune your agents to deliver exceptional value—allowing you to charge premium pricing and establish a strong market position. In My AI Agents Training, you will build, lots and lots of specialized AI Agents in **Healthcare, Finance, Aviation** and more.

26 Agents Don't Replace People



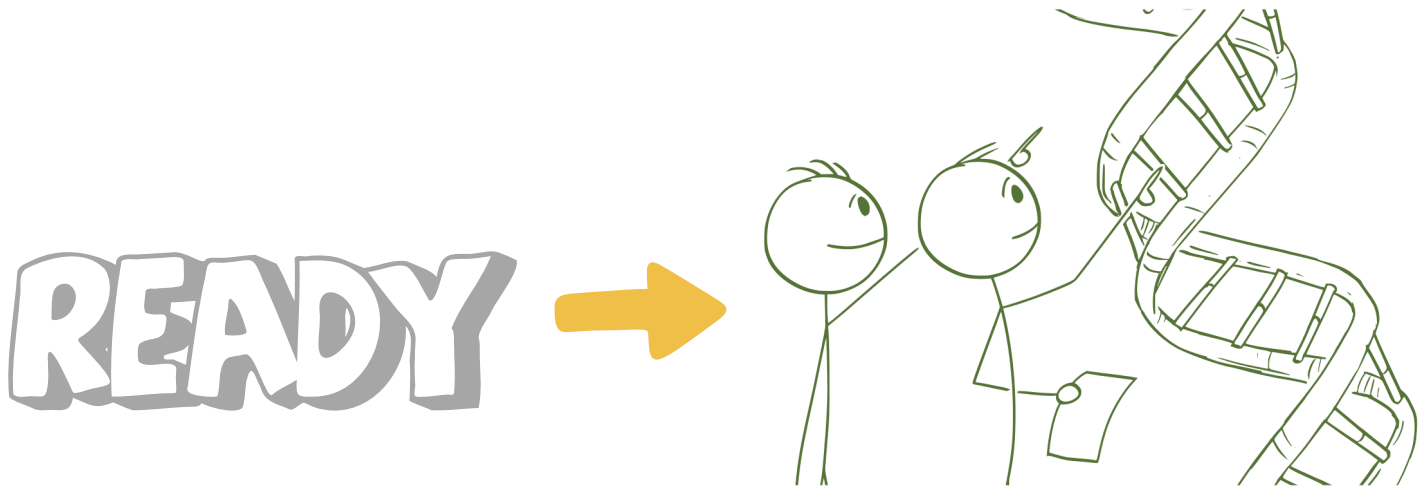
Perhaps the most reassuring lesson is that AI agents are not here to replace human workers. Instead, they enable businesses to scale operations, improve efficiency, and free up employees to focus on higher-level, more fulfilling tasks. Rather than triggering job cuts, AI agents empower teams to think bigger, innovate faster, and achieve greater revenues and profits. Humanity will always have a vital role, even in an increasingly automated world. If you like a positive take on AI Future watch this **TED Talk** of Thomas Wolf, Hugging Face.

27 Avoid Overcomplicating Prompt Engineering



You can easily overcomplicate prompt engineering by using excessively long, complex prompts without clear instructions or step-by-step guidance—simply hoping that the LLM will figure everything out. For example, when training AI agents for healthcare in a hospital, rather than listing every possible instruction for a doctor, keep the initial prompt simple (e.g., “Your experienced doctor, start a conversation with a patient and give symptoms and diagnosis”). Then, support the LLM with real-world data such as tabular data from Kaggle containing actual patient measurements or even leverage specialized medical imaging models (e.g., for pneumonia detection from X-ray images via Hugging Face). In this way, you don’t leave everything to the LLM, but rather use it to bring together structured, domain-specific inputs. BTW, if you want to learn about **Hugging Face Watch this Free Hands-on tutorial**.

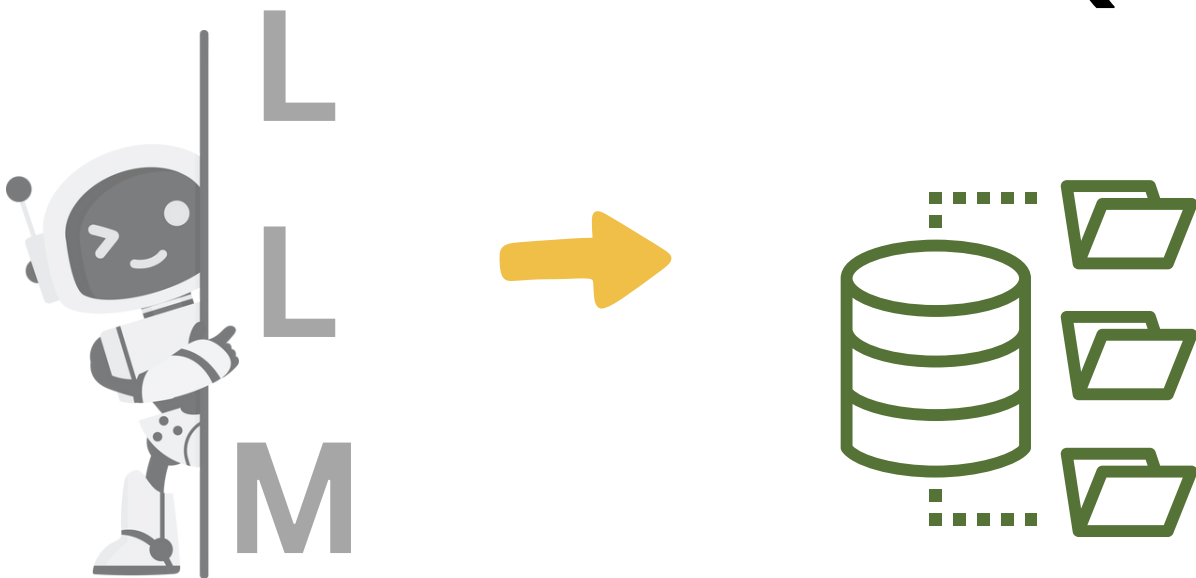
28 **Test Your AI Tools & Agents Separately**



Another common pitfall is not testing your individual tools and agents independently. Instead of simply integrating data sources, web search, function calls, and image processing components all at once, ensure each element is verified on its own first. By testing every tool and agent separately—much like validating each Python function in a data science project—you guarantee that when they are finally combined into the complete graph, they work seamlessly together. This practice saves you from later having to trace through complicated code to diagnose issues. This has been my method in all my Trainings.

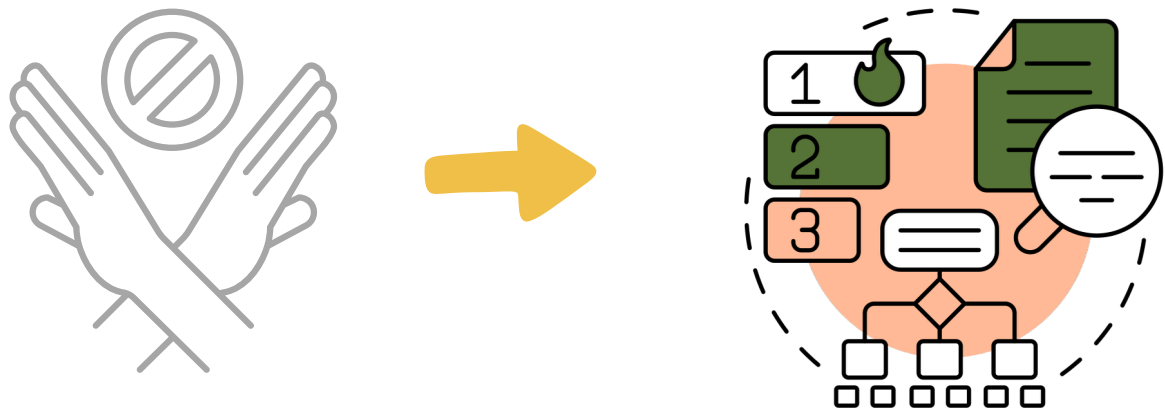
29

Leverage Retrieval Augmented Generation (RAG)



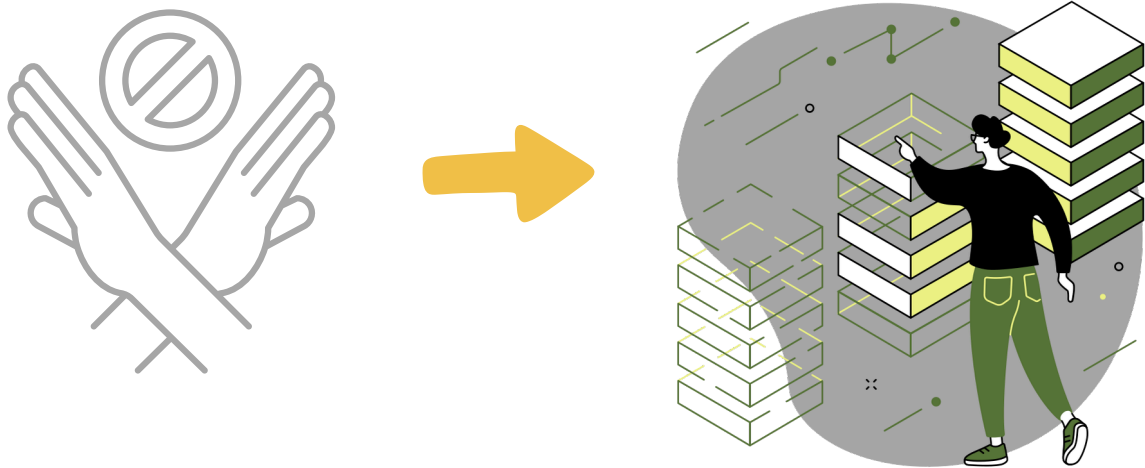
LLMs are prone to hallucinations or providing outdated, non-transparent information. To counter this, incorporate Retrieval Augmented Generation (RAG). With RAG, the LLM retrieves relevant pieces and snippets from documents (using semantic similarity and techniques such as embedding and chunking), augments the retrieved data for improved quality, and then generates an accurate response. This method not only enhances transparency and accuracy but also secures your internal documents by keeping sensitive information off the public internet. Building a good RAG pipeline and its fundamental is part of My **training: AI Agents Mastery - Complete**.

30 Respect Token Limits and Optimize Context Windows



Ignoring token limits and context windows is a recipe for failure. Feeding huge chunks of text—such as entire documents or datasets—into an LLM can easily exceed its token capacity, causing errors or incomplete processing. Instead, break your data into manageable clusters (for example, monthly segments or by specific categories) so that each chunk retains its context while staying within the model's limits. This approach ensures that your AI agent processes data accurately and efficiently. In my **CrewAI AI Agents Training** I use Financial Data and break it into monthly data.

31 Leverage Vector Databases and Embeddings Effectively



Not using embeddings and vector databases correctly can drastically influence the retrieval performance of your AI agents. The quality of your embeddings, the chosen vector database (such as FAISS or Chroma), and even the optimal chunk size all play crucial roles in the end result. Fine-tuning these parameters—based on your project’s needs—ensures that the AI agent retrieves the most relevant information without diluting context or missing key details. The right Embedding can change the quality of your outcome. See how this effects a **Local RAG for Nvidia Complex Documents** in this free Tutorial.

32

Choose the Right LLMs and Platforms for Task Specialization



A one-size-fits-all approach with LLMs can lead to subpar results. Different models excel at different tasks—what works well for creative tasks (like Mistral) may not be ideal for precise reasoning. When building specialized agents (such as a gender classification model or a medical imaging classifier), choose platforms and models that have been pre-trained with the right data. Explore Hugging Face's extensive library or platforms like Groq and Ollama, which offer models tailored for specific tasks and optimized for performance. For example, the latest DeepSeek is built differently. I explained **DeepSeek Building Stones** in this blog post.

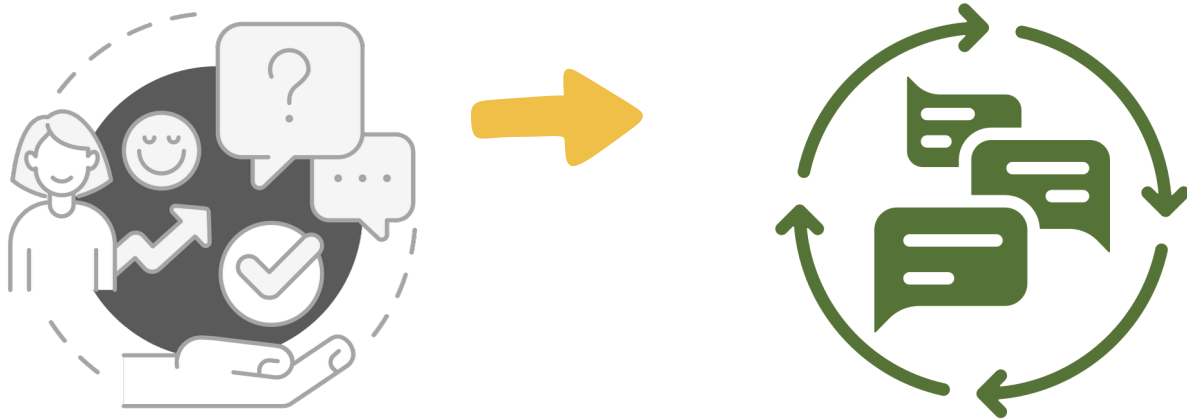
33

Prioritize Performance Optimization



Skipping performance optimization can severely degrade user experience. No one wants to wait twenty minutes for an AI agent's response. Invest in performance optimization techniques—such as tensor streaming architectures (as demonstrated by Groq), speculative decoding, and automated model compilers—to boost speed and reduce latency. Additionally, leverage hardware utilization strategies (including model quantization and fine-tuning) to ensure that your agents respond quickly and reliably. Groq + Llama (Open-source LLM) is Powerhouse, I use this in a **Hospital Project here.**

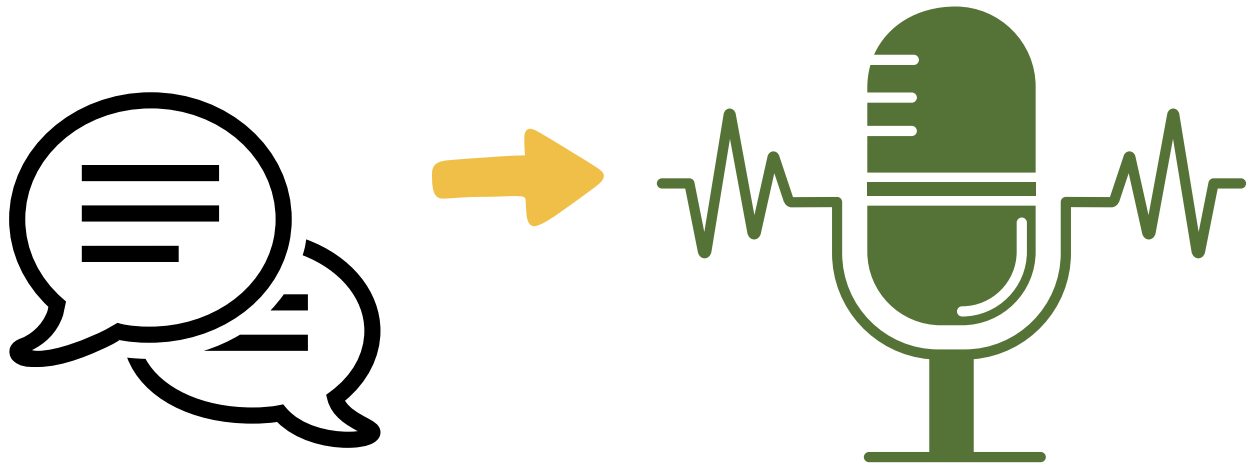
34 Avoid Relying on LLMs for Repetitive Mathematical Tasks



LLMs are not inherently designed for tasks like counting or repetitive arithmetic, which can lead to infinite loops or inconsistent results. For instance, if an agent responsible for passport control in an airport must count individuals, relying on the LLM for counting may result in errors. Instead, delegate repetitive mathematical tasks to simple programming constructs (e.g., a Python loop) and use context variables to pass sequential data to the LLM. This division of labor helps maintain accuracy and efficiency. In **my LangGraph training** I am building a Passport Control for an Airport with Vision Models and I show case this.

35

Incorporate Text-to-Speech (TTS) for Enhanced User Experience



Ignoring text-to-speech capabilities means missing out on adding a human-like touch to your AI agents. Integrating TTS not only enhances the interactive experience but also serves as a finishing touch that makes your application feel complete. Platforms like ElevenLabs are highly regarded, and there are also open-source alternatives (such as PlayIT or Coco TTS from Hugging Face) that support multiple languages. Adding TTS functionality can significantly improve the usability and appeal of your AI solution. For building TTS AI Agents in a few minutes **Watch this Free Tutorial.**

36

Utilize Context Variables and Dynamic Prompt Templates

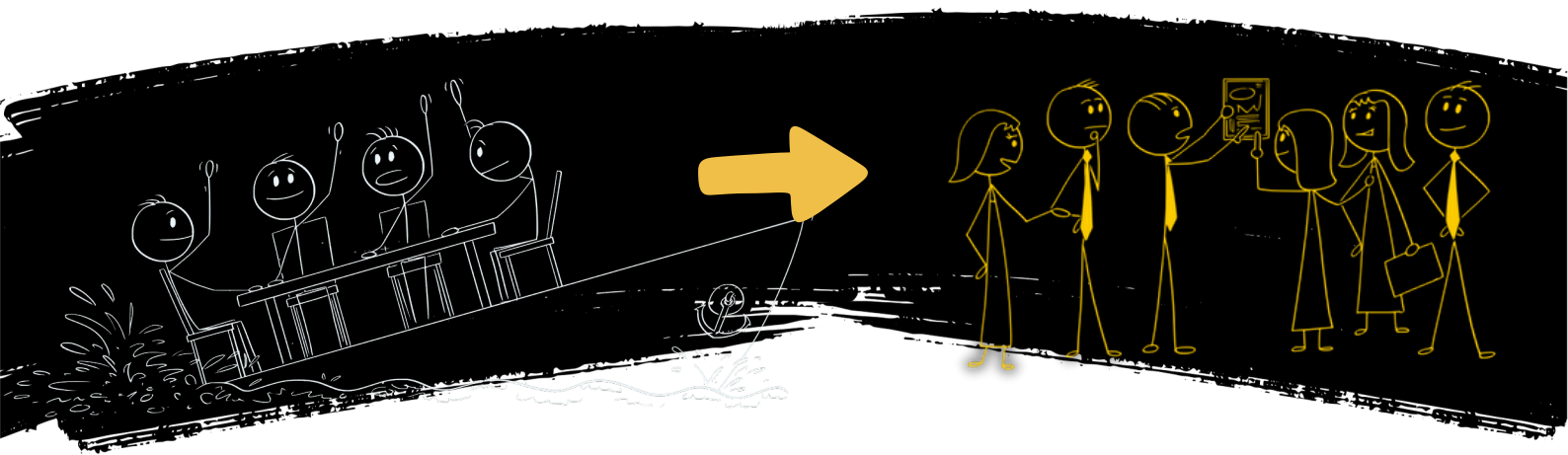
Chat



Hardcoding prompts into your code is inefficient and inflexible. Instead, leverage prompt templates and context variables to dynamically adjust your prompts. This means you can pass in dynamic information—such as candidate names or vacancy details for a contact agent—without having to rewrite the prompt for every version. This approach not only saves time but also makes your AI agents more adaptable and easier to maintain across diverse scenarios. This is one of the Tips in this **Free Hands- on Tutorial**.

37

Leverage LangGraph & CrewAI for Flow Engineering



Effective AI agent development often requires a clear, step-by- step approach—much like preparing a cup of coffee by thinking through each action (grabbing the mug, placing it under the coffee machine, and pressing the button). Leveraging flow engineering frameworks such as LangGraph allows you to visualize tasks as interconnected nodes and edges, ensuring that each step is executed in sequence. This structured approach minimizes mistakes and streamlines the overall development process. In the **CrewAI part of AI Agents Mastery**, I show you how to build Flow Engineering for Finance.

38 Incorporate Dense Passage Retrieval (DPR) Alongside RAG

RAG



DPR

CAG

GraphRAG



While RAG is essential for mitigating hallucinations and retrieving relevant information, many overlook the benefits of Dense Passage Retrieval (DPR). DPR uses dense vector representations to rapidly extract the most pertinent passages from large corpora, ensuring faster and more accurate retrieval. By combining DPR with RAG, you can further focus on retrieving exactly what's necessary for your AI agent, ultimately speeding up response times and reducing errors. I show you how to build DPR in this **Free Hands-on Tutorial** for Videos.

39

Write Unit Tests for Each Component

Break down your agent into testable components and use unit tests to validate behavior. For instance, test your prompt templates, chain logic, and individual function calls using a framework like pytest. In my **Training**, you will learn to build and test each agent / tool first.

```
def test_prompt_template():
    prompt = template.format(doctor_name="Jones", patient_symptoms="headache")
    assert "Jones" in prompt
    assert "headache" in prompt

def test_data_processing_node():
    result = processing_node.function("test input")
    assert result == "TEST INPUT"

if __name__ == "__main__":
    test_prompt_template()
    test_data_processing_node()
```

40

Visualize Agent Flows with LangGraph

LangGraph's Nodes and Edges let you design agent workflows visually and programmatically. The above code is from my hands-on AI Agents Training, **AI Agents Mastery**.

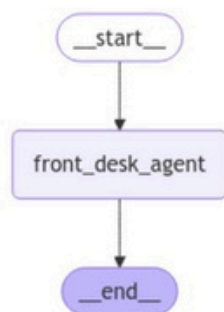
```
front_desk_agent_node = functools.partial(front_desk_agent,
                                           image_Path = image_Path,
                                           selected_patient_data=selected_patient_data,
                                           updated_patient_df =updated_patient_df)

# 6. Set up the Langgraph state graph
FrontDeskGraph = StateGraph(AgentState)

# Define nodes for workflow
FrontDeskGraph.add_node("front_desk_agent", front_desk_agent_node)
FrontDeskGraph.add_edge(START, "front_desk_agent")
FrontDeskGraph.add_edge("front_desk_agent", END)

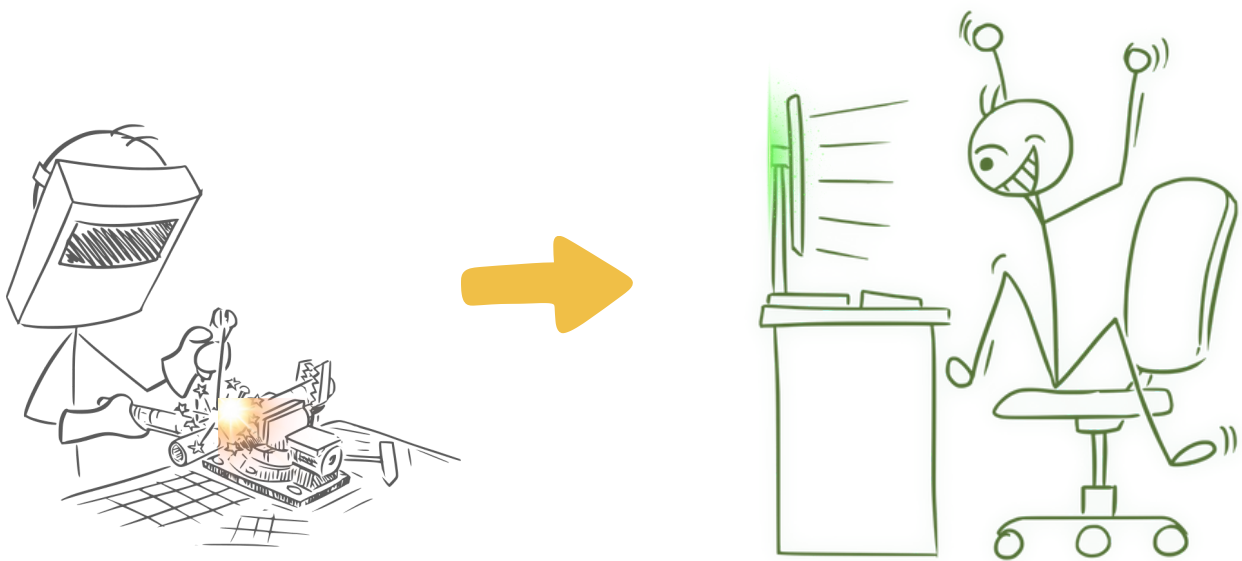
# Initialize memory to persist state between graph runs
FrontDeskWorkflow = FrontDeskGraph.compile()

from IPython.display import Markdown, display, Image
display(Image(FrontDeskWorkflow.get_graph(xray=True).draw_mermaid_png()))
```



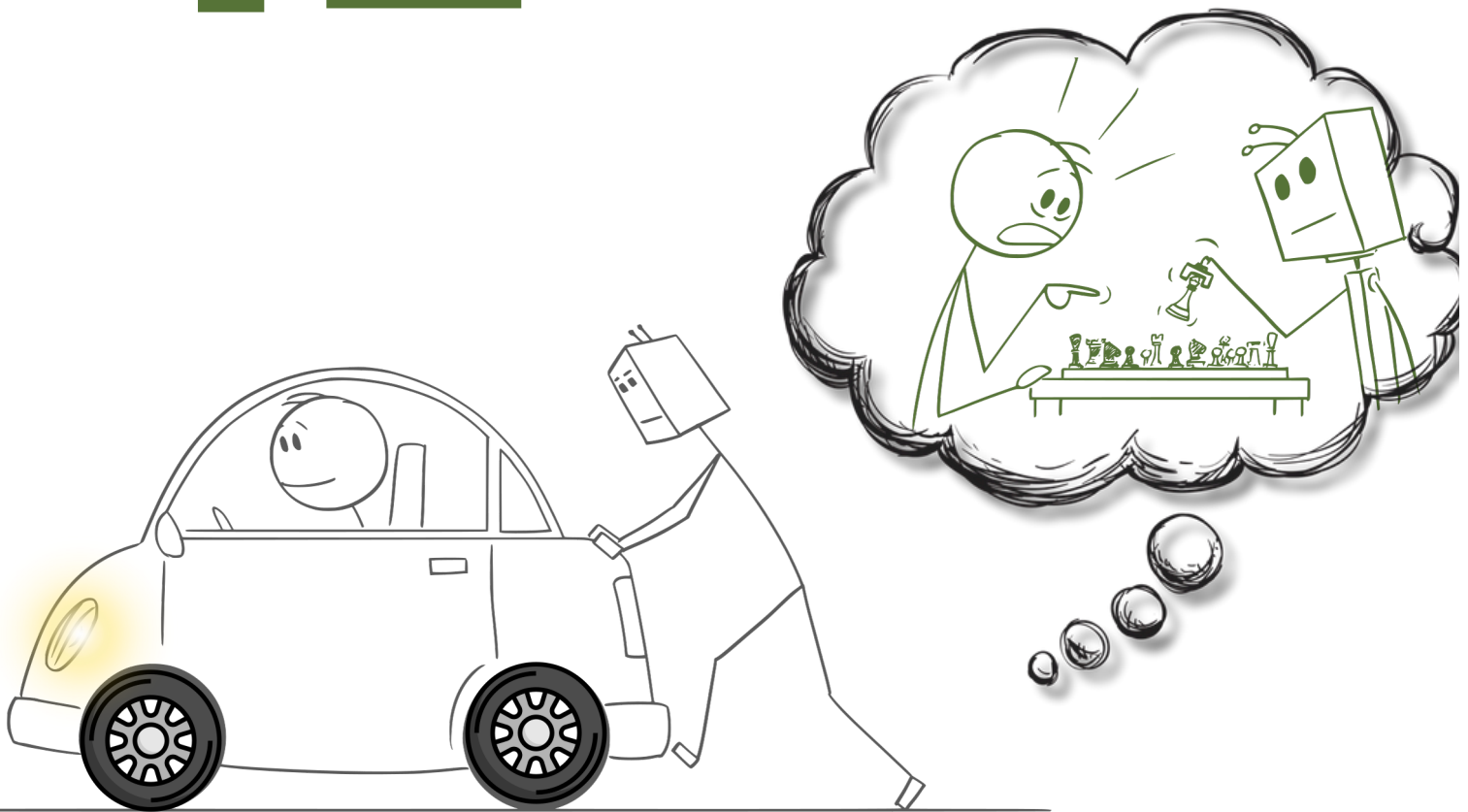
41

Be Empathetic to the Model's Perspective



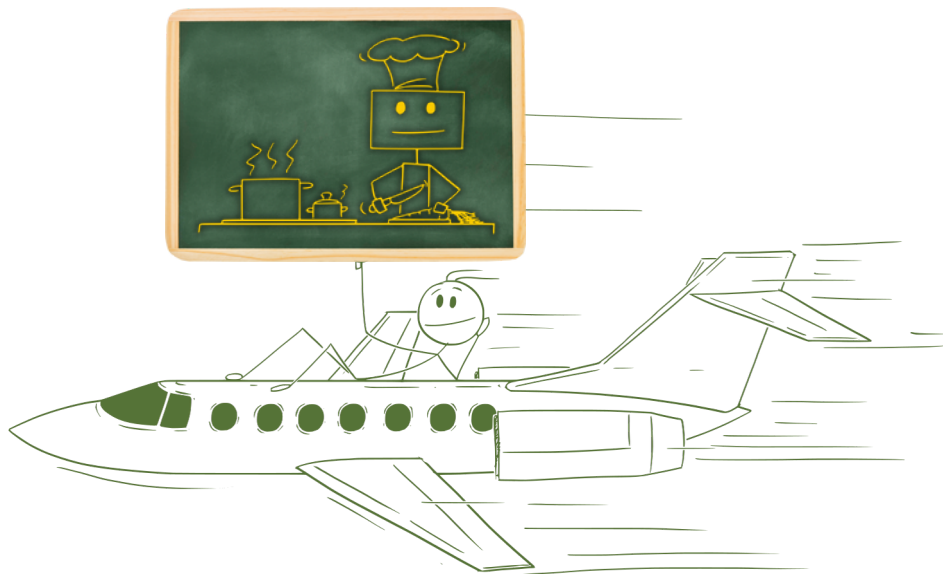
Being empathetic to the model means intentionally stepping into the AI's shoes to anticipate its gaps and misunderstandings. Models lack real-world experiences, implicit knowledge, and common sense you naturally use. They rely entirely on what's explicitly defined or described. If a prompt or a tool's documentation is unclear, incomplete, or ambiguous, the AI agent may misinterpret the intent, leading to incorrect outcomes or unexpected behaviors. By clearly understanding what the model knows and doesn't know, you increase the likelihood that it performs tasks predictably and consistently. Watch

42 when should you use AI Agents?



AI agents are particularly effective for complex, open-ended tasks where the number of steps cannot be predetermined and flexibility is essential. They are well-suited for scenarios requiring dynamic decision-making, such as customer support interactions and coding tasks, where the agent can autonomously plan and execute actions based on real-time feedback. However, it's crucial to balance their autonomy with appropriate guardrails and thorough testing to mitigate risks associated with higher costs and potential error accumulation. In contrast, for tasks that can be clearly defined and decomposed into fixed subtasks, implementing structured workflows with predefined code paths may be more efficient and cost-effective. Read **Anthropic view** on this.

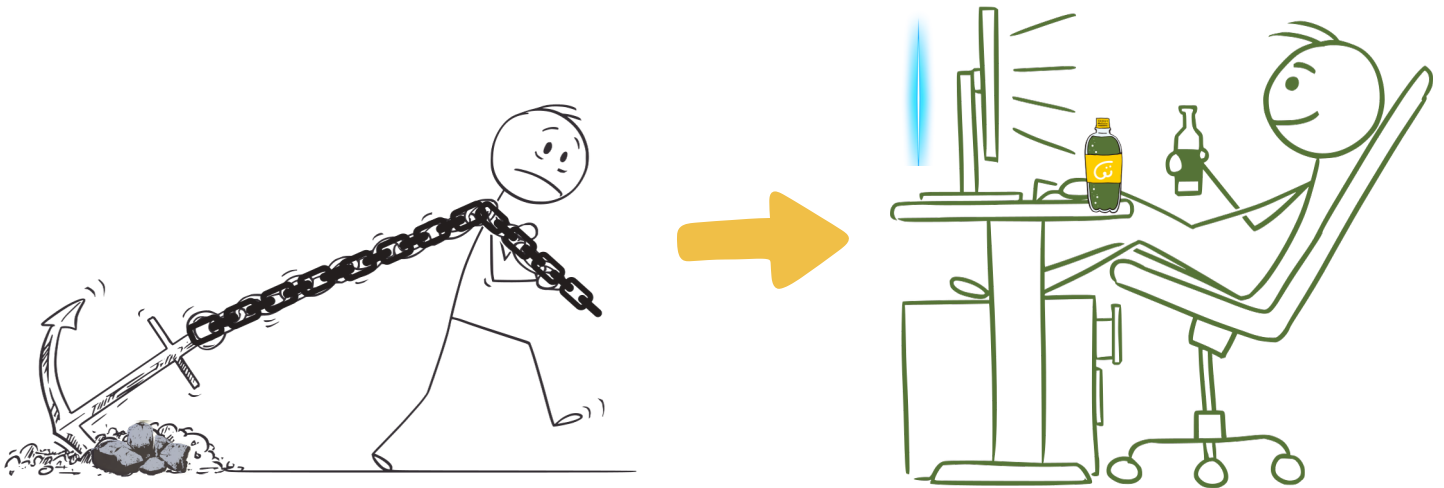
43 Consumer-Oriented Agents Are Currently Overhyped



current consumer-oriented agents may be overestimated in their capabilities. Tasks such as fully automating a complex vacation booking require intricate understanding of personal preferences, real-time data, and nuanced decision-making. Current AI agents often lack the depth of context and the ability to verify details accurately, making the automation process as labor-intensive as manual booking. For consumer-focused agents to become truly effective, they need to incrementally build and understand user-specific contexts over time, allowing for more personalized and accurate assistance. Planning a vacation involves numerous personal preferences and contextual factors—such as budget constraints, preferred destinations, activity interests, and accommodation types—that are difficult for AI agents to fully comprehend and integrate without extensive user input.

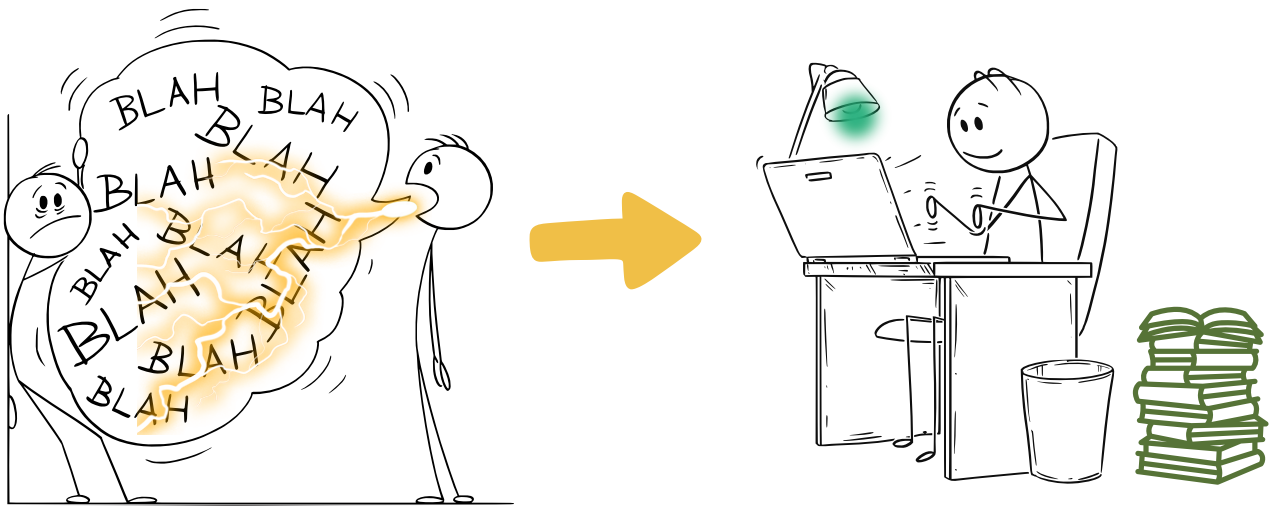
44

Pydantic AI Simplifies Building AI agents



Pydantic AI simplifies building AI agents by using structured data classes, making input/output management and dependency handling straightforward. It seamlessly integrates multiple search engines like DuckDuckGo and Tavily, and supports asynchronous queries for faster, concurrent searches. Agents dynamically adapt the search depth and formatting of outputs based on flexible system prompts, allowing real-time injection of variables such as dates to ensure relevant and timely results. Structured outputs enable precise formatting of responses into useful components, such as markdown summaries and bullet points, improving clarity and usability. Providing examples in ready-to-use notebooks further accelerates experimentation and customization, facilitating immediate hands-on exploration without complicated setups. Both **Langchain** and **Pydantic AI** use Pydantic approach.

45 Context Engineering is AI Agents Feature Engineering

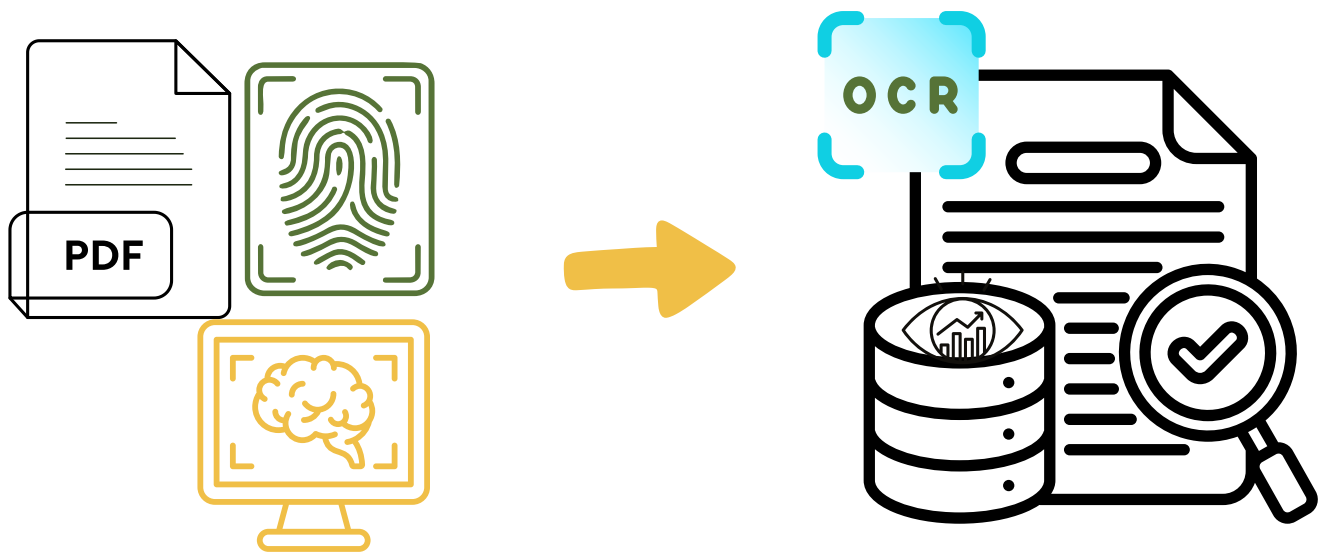


Simply increasing context window size doesn't guarantee better performance. Beyond 128K-200K tokens, agents experience "context rot" where retrieval accuracy degrades, tool selection becomes confused, and response quality collapses despite having capacity remaining. This Pre-Rot Threshold occurs because context capacity $\neq$ context quality. Production systems require strategic context engineering: Layered Action Spaces that support infinite capabilities through focused 10-function interfaces, Context Offloading via scratchpads and external memory, the Compaction Principle for reversible compression (versus irreversible summarization), Agent-as-Tool patterns that hide complex workflows behind clean schemas, Context Quarantine through multi-agent isolation, and Dynamic Tool Selection using RAG to reduce confusion.

[Watch this Video.](#)

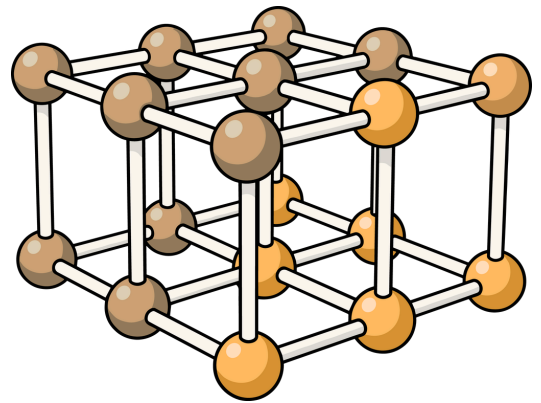
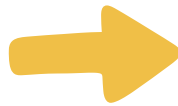
46

Document Parsing Quality Determines Agent Accuracy



Poor document parsing is the silent killer of AI agent performance. When agents receive garbled PDFs, misaligned tables, or text with broken formatting, even the best LLMs produce incorrect outputs. Traditional OCR tools often mangle complex documents—converting tables into unreadable text streams, losing hierarchical structure, and scrambling multi-column layouts. Production systems require robust document parsing: Tools like Docling, PyMuPDF4LLM, and Marker preserve document structure, maintain table integrity, and handle complex layouts. For scanned documents, modern OCR solutions like PaddleOCR, Tesseract 5, and cloud-based services (Google Document AI, AWS Textract) extract text with high accuracy while preserving spatial relationships. The key insight: garbage in, garbage out. [Watch this video](#)

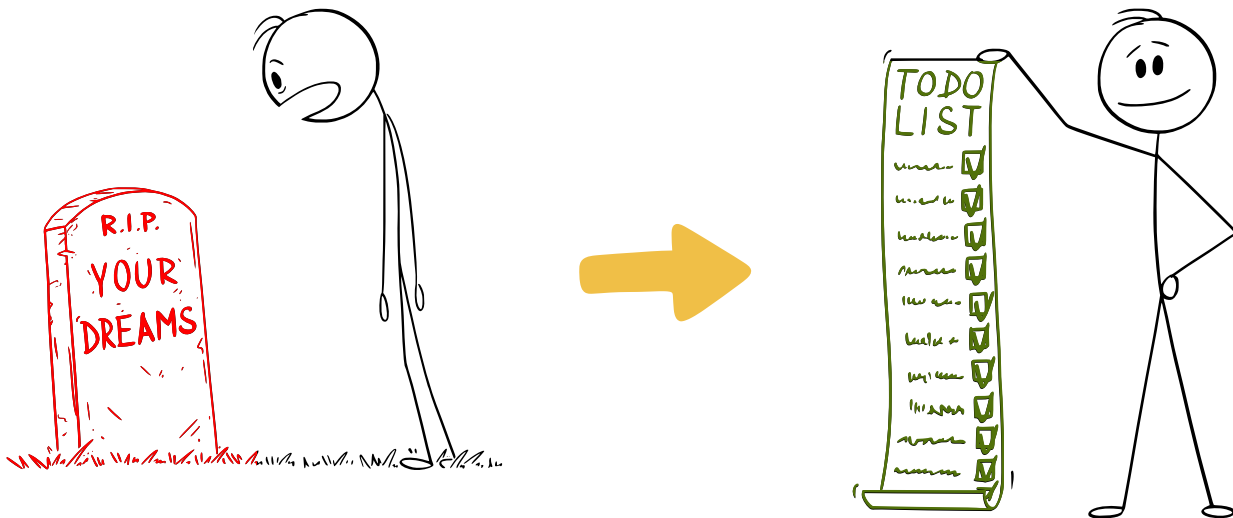
47 The No-Anxiety AI Agents are Systems



The anxiety of building AI agents is real, but it's not coming from where you think. Every morning brings a new framework, a new "revolutionary" model, and someone declaring that last month's work is obsolete. Even capable engineers experience imposter syndrome—not from lack of skill, but because the goalposts keep moving. Decision paralysis sets in: LangGraph or CrewAI? OpenAI SDK or PydanticAI? GPT, Claude, or Gemini? Here's the truth: most agent frameworks are just different libraries around the same core ideas—define a role, structure inputs and outputs, add reasoning, call tools, store memory, evaluate results. The models? Comparable capabilities with slight trade-offs. The real mistake is thinking you must pick "the right one" before you can start. You don't. Start with one. Build something real. Then expand. Switching later is much easier than people think. When you view agents through stable building blocks (role and goal, structured I/O, reasoning loops, tool use, memory, evaluation), framework churn becomes noise. AI moves fast. Reliable systems move much slower.

48

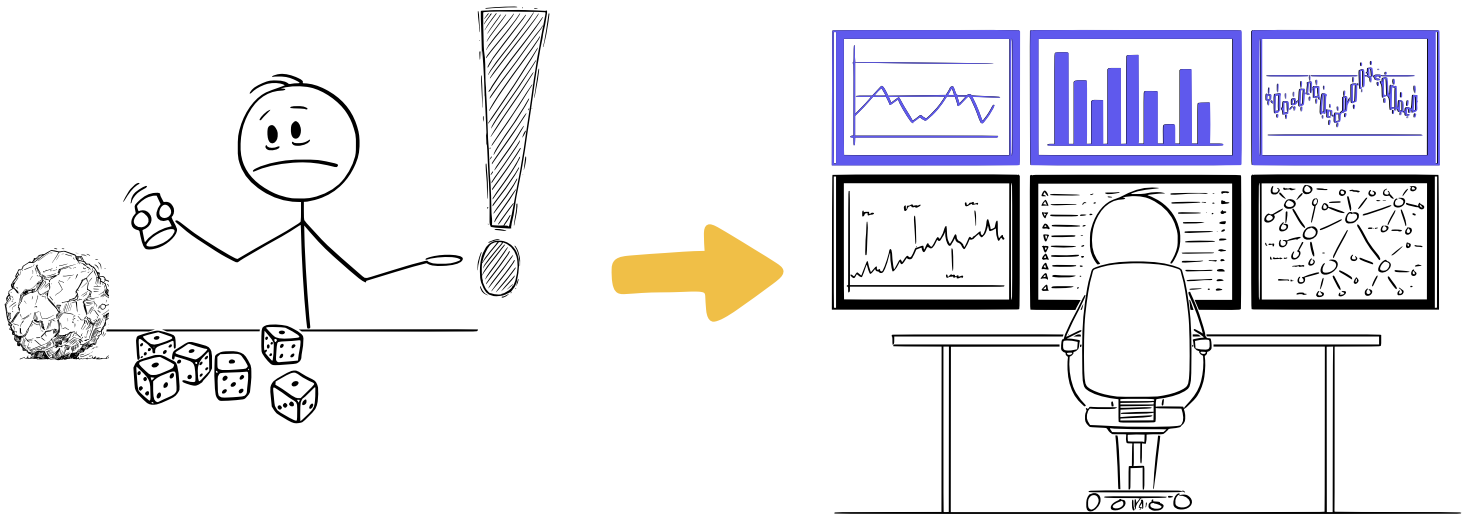
Mitigate AI Agent Hallucinations



LLMs hallucinate. AI agents compound hallucinations across reasoning, tools, and memory—turning single errors into cascading failures. Production systems require systematic mitigation strategies across five critical layers: Verification Architecture (Chain-of-Verification generating validation questions, Parent-Child topology with dedicated critics, Blind Critics reviewing outputs without seeing reasoning, Debate patterns where agents argue contradictory positions, Cross-Model verification using small models to draft and SOTA models to verify), Structural Constraints (enforcing JSON schemas where parse failures signal hallucinations, Pre-Check validation of API arguments against schemas, Negative Constraints explicitly defining forbidden outputs, Sanity Checks with hard-coded bounds, Protocols standardizing agent communication), Memory and Context (Reflexion storing past errors to prevent repeats, Context Cleaning through frequent summarization, Grounding by observing state before acting, Clean Context maintenance as long histories breed errors), Multi-Agent Patterns (Specialization with SQL + Python agents beating generalists, Isolation where critics send feedback without mixing roles, Editor Patterns with dedicated agents removing unverified claims, RAG Critics with "Truth" databases creators lack), and Runtime Validation (Watchdogs using non-LLM scripts for validation, Fractal Sampling querying 3x where variance flags hallucinations, Scoring systems discarding outputs below thresholds, Voting across instances, Kill Switches stopping after failed retries, Sensitivity testing where punctuation changes reveal instability).

49

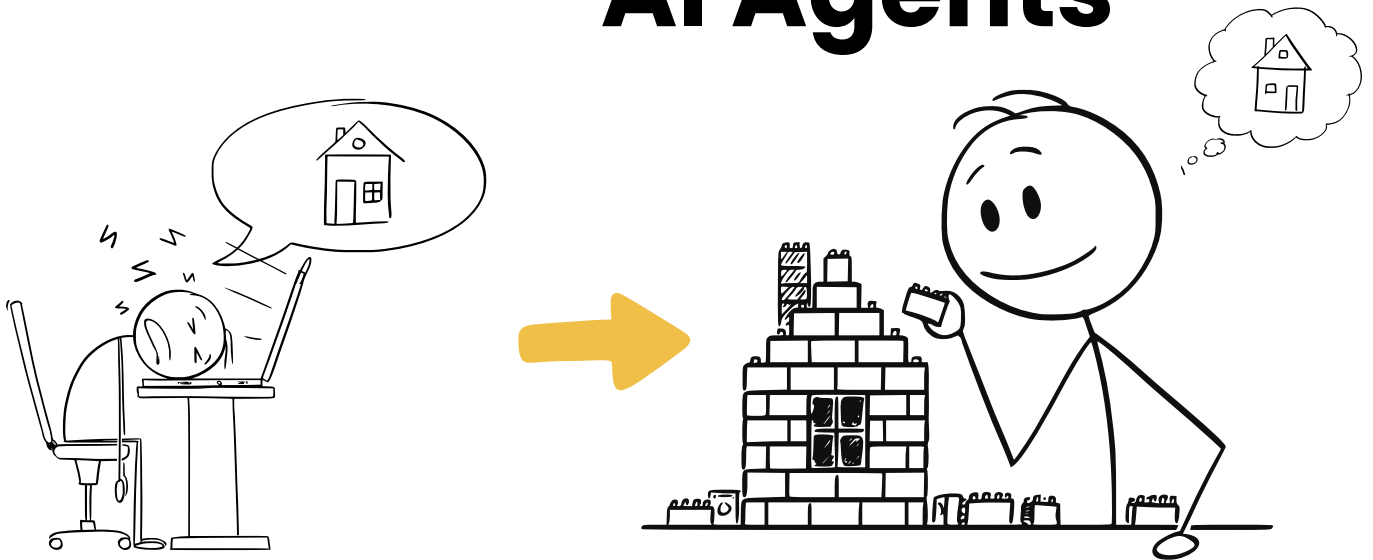
Governable AI Agents is a Must



An AI agent is a goal-driven control system, not a conversational feature. The recurring assumption is that better reasoning compensates for weak structure, adding memory automatically improves decisions, and tool access equals capability. In production, the opposite happens. When perception, reasoning, planning, and execution are not explicitly separated, the agent becomes ungovernable—you cannot trace decisions, constrain behavior, or intervene safely. Governable agents are built by reducing freedom and creating boundaries. In one word: architecture. Architecture creates predictability. Predictability creates trust. Production systems follow seven governance rules:

- ① Lock the decision boundary: define exactly what the agent may optimize and what it must never decide; if it's not a valid action type, it cannot execute.
- ② Split thinking from acting: one path plans, another executes; execution never accepts free-text.
- ③ Treat memory as risk: persist only validated state, ephemeral by default, forget aggressively.
- ④ Add a reflection gate: recheck assumptions, compare against prior state, retry or stop.
- ⑤ Assign single ownership: one agent, one responsibility, structured handoffs only.
- ⑥ Design for observability: log inputs, decisions, outcomes; if you can't trace it, don't ship it.
- ⑦ Optimize for trust: predictable behavior beats impressive outputs every time.

50 Learn AI Agents by Building AI Agents



If AI agents still feel confusing after 100 tutorials, it's because passive learning doesn't build the mental models needed for production work. The "Worth Its Weight in Gold" roadmap has three stages: ① Learn the Fundamentals—understand how LLMs work, what system prompts are, what memory is, basic Python, and I/O formats (JSON, Markdown). Use Google, Claude, or ChatGPT to ask for sample code. Fundamentals create the foundation, but they're not enough alone. ② Learn Different Frameworks in Python—frameworks don't matter until fundamentals are solid, then they matter a lot. If you're comfortable with Python: LangGraph → PydanticAI. If your Python level is more basic: OpenAI Swarm/SDK → CrewAI. Free vs paid? The real difference is learning speed. Choose free options where building is central, not frameworks trying to sell you tools. Paid options make sense if they fast-track learning and combine multiple frameworks. ③ Start Building by Projects—this is where everything clicks. Projects turn confusion into intuition. Learn by doing. The more you build, the more you learn. Build across different industries, especially ones that inspire you: healthcare, finance, supply chain, engineering. Build diverse agents like geo, traffic, air-quality agents. Tutorials teach syntax. Projects teach systems thinking, debugging, trade-offs, and production patterns. The confidence comes from Building. My 2,300 students build real-world AI agents.

FREE TRAINING

How to Build Real-World AI Agents in 30 Minutes

Zero to Hero Deep Dive

Free for a limited time.

 **FREE BONUS:**

Unlock the training and get a 56-page guide with battle-tested AI Agents strategies!



Dr. Maryam Miradi



Chief AI Scientist
PhD | 20+ years in AI
Educator in 90+ Countries

What you'll Learn:

- ✓ What AI Agents really are and why you shouldn't treat them like chatbots
- ✓ The 4 key components every serious agent must include
- ✓ How to build AI Agents with the best frameworks (LangGraph, PydanticAI, CrewAI, Swarm)
- ✓ How to move from code demo to Real-World AI Agents
- ✚ How to keep up with AI advances and stay focused in an overwhelming AI landscape
- 🔊 Includes English captions: perfect for watching anywhere, anytime



About Author

Dr. Maryam Miradi



- 
- * Chief AI Scientist PhD in
 - * AI 20+ years AI
 - * Experience AI Expert
 - * International Educator
 - * in 90+ Countries



Chief AI Scientist, CEO
and Founder of Profound
Analytics in Amsterdam



Author of 2 Books, 24
research papers, holder of
5 international awards,
including Best European
Researcher in Future Vision



a PhD in Artificial Intelligence



20+ years developing AI Models
and solutions, leading 200+
projects across 12 industries
(finance, energy, retail,
healthcare, and beyond)



International Educator and
trained thousands of AI
professionals in 90+
countries across all
continents



www.maryammiradi.com



Ready to Build Real AI Agents That Actually Work?

**Join 2,300+ Professionals in My
AI Agents Mastery 5-in-1 Training**

Clickable Link

www.maryammiradi.com



SCAN
ME



Excellent ★★★★★

Trusted by 2,300+ AI
Professionals Worldwide

Instructor

Dr. Maryam Miradi

20+ Years in AI



AI Agents 50 Best Practices

★★★★★ (5/5) — 500+ Copies Sold
AI pros are loving this stunning book.

Build Real-World AI Agents That Actually Work

50 AI Agent Tactics. One Gorgeous Book. Zero Fluff.



 Now Signed by the
Author
Dr. Maryam Miradi

Get Limited Edition

